

UNIT - V

5

Turing Machines (TM)

Syllabus

Turing Machine Model, Formal definition of Turing Machines, Language Acceptability by Turing Machines, Design of TM, Description of TM, Techniques for TM Construction, Computing function with Turing Machine, Variants of Turing Machines, Halting Problem of TM, Halting vs Looping, A Turing-unrecognizable language, Reducibility, Recursion Theorem. The Model of Linear Bounded Automata.

Contents

- 5.1 Turing Machine Model
- 5.2 Design of TM **Dec.-13**, Marks 3
- 5.3 Instantaneous Description(ID) of TM **Dec.-17, May-18**, Marks 4
- 5.4 Techniques for TM Construction **May-19**, Marks 8
- 5.5 Language Acceptability by Turing Machines
..... **Dec.-06,07,09,11,13,14,17,18,19**,
..... **May-06,11,13,14,16,18,19**, Marks 10
- 5.6 Computing Function with Turing Machine **Dec.-05,07,08,09,10,15,17,18,19**,
..... **May-06,07,10,12,14,15,16**, Marks 12
- 5.7 Variants of Turing Machines..... **Dec.-17**, Marks 4
- 5.8 Halting Problem of TM **May-16,19, Dec.-17,19**, ... Marks 4
- 5.9 Halting Vs Looping
- 5.10 A Turing - unrecognizable Language
- 5.11 Reducibility
- 5.12 Recursion Theorem
- 5.13 The Model of Linear Bounded Automata
- 5.14 Fill in the Blanks
- 5.15 Multiple Choice Questions

5.1 Turing Machine Model

The Turing machine is a collection of following components.

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \Delta \text{ or } B, F)$$

- 1) Q is a finite set of states.
- 2) Γ is finite set of external symbols.
- 3) Σ is a finite set of input symbols.
- 4) Δ or b or $B \in \Gamma$ is a blank symbol majorly used as end marker for input.
- 5) δ is a transition or a mapping function.

5.2 Design of TM

SPPU : Dec.-13, Marks 3

The Turing machine can be modelled with the help of following representation.

- 1) The input tape having infinite number of cells, each cell containing one input symbol and thus the input string can be placed on a tape. The empty tape is filled by blank characters.

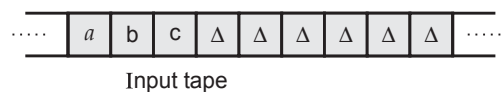


Fig. 5.2.1 Input tape

- 2) The finite control and the tape head which is responsible for reading the current input symbol. The tape head can move to left or right.
- 3) A finite set of states through which machine has to undergo.
- 4) Finite set of symbols called external symbols which are used in building the logic of turing machine.

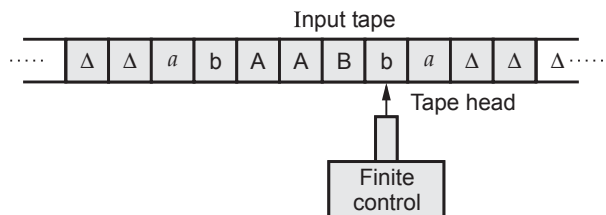


Fig. 5.2.2 Turing machine

Review Question

1. Give formal definition of move relation of turing machine.

SPPU : Dec.-13, Marks 3

5.3 Instantaneous Description(ID) of TM

SPPU : Dec.-17, May-18, Marks 4

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \Delta \text{ or } B, F)$ be TM then the Instantaneous Description (ID) of a TM can be given by a **triplet** or a **program** which is as given below -

$$(q_0, a) \rightarrow (q_1, A, L)$$

That means currently we are reading the input symbol 'a' and we are in q_0 state then we can go to q_1 state by replacing or printing 'a' by A and then we must move in **left** direction.

Review Question

1. Explain the representation of TM.

SPPU : Dec.-17, May-18, End Sem, Marks 4

5.4 Techniques for TM Construction

SPPU : May-19, Marks 8

Construction of turing machine is a process of writing out the complete set of states and next move function. This is a totally conceptual phenomenon. The turing machine can be designed with the help of some conceptual tools. Let us discuss some of these tools.

5.4.1 Storage in Finite Control

As shown in Fig. 5.4.1, the model of turing machine has a finite control. This finite control can be used to hold some amount of information. The finite automata stores the information in pair of elements such as the current state and the current symbol pointed by the tape head. This is just a conceptual arrangement.

For example - The function δ can be written as follows.

$$\delta([q_0, 0], 1) \rightarrow ([q_1, 1], \Delta, R)$$

This means that if finite control shows the initial state is q_0 and stores the current symbol 0 if it reads the symbol 1 then the machine goes to next state q_1 replace that 1 by Δ and moves to right. This helps in building the transition graph of the language.

Example 5.4.1 Construct a turing machine M for $\Sigma = \{a, b\}$ which will convert lower case letters to upper case.

Solution : The turing machine is designed such that we will get the equivalent upper case string to the input. The q_0 is a initial state and q_3 is final state. The δ function is written along each edge. For the sake of understanding consider the transition between q_0 and q_1 , if the finite control reads on the tape A will be printed there and the move will be in right direction. After converting the given string to upper case we reach to the final state q_3 . The q_3 is a halt state.

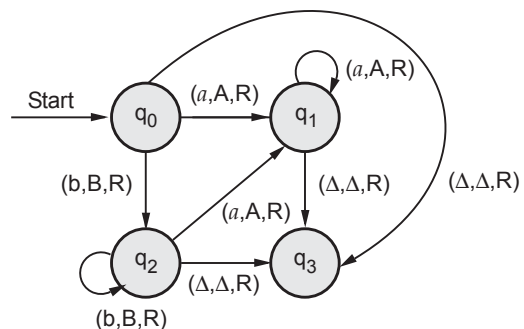


Fig. 5.4.1

5.4.2 Multiple Tracks

If the input tape is divided into multiple tracks then the input tape will be as follows -

For example :

As shown in Fig. 5.4.2 the input tape has multiple tracks on the first track. The input which is placed is surrounded by # and \$. The unary number equivalent to 5 is placed on the input tape, on the first track. On the second track unary 2 is placed. If we construct a TM which subtracts 2 from 5 we get the answer on the third track and that is 3, in unary form. Thus this TM is for subtracting two unary numbers with the help of multiple tracks.

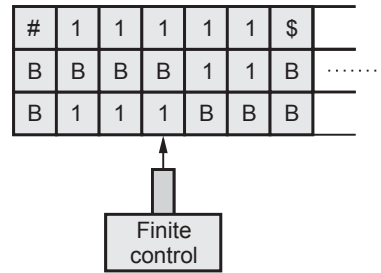


Fig. 5.4.2 Multiple tracks

5.4.3 Checking Off Symbols

Checking off symbols is an effective way of recognizing the language by TM. The symbols are to be placed on the input tape. The symbol which is read is marked by any special character. The tape head can be moved to the right or left. Let us take some example and we will see how to build turing machine by checking of symbols.

Example 5.4.2 Construct a turing machine $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ which recognizes the language $L = \{w c w \mid w \in (a + b)^+\}$.

Solution : In this language the input set is $\Sigma = \{a, b\}$. The string which when will be placed on the input tape it will have two distinct parts separated by letter c, such as

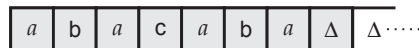


Fig. 5.4.3

In the checking off symbols, each symbol is marked by special character. The simple logic in construction of this TM will be we mark the first letter and then move to the right till we not get c, the first letter after c will be compared with the marked letter. If it is the same as which we have marked then mark this symbol otherwise goto reject state. It can be shown as below,

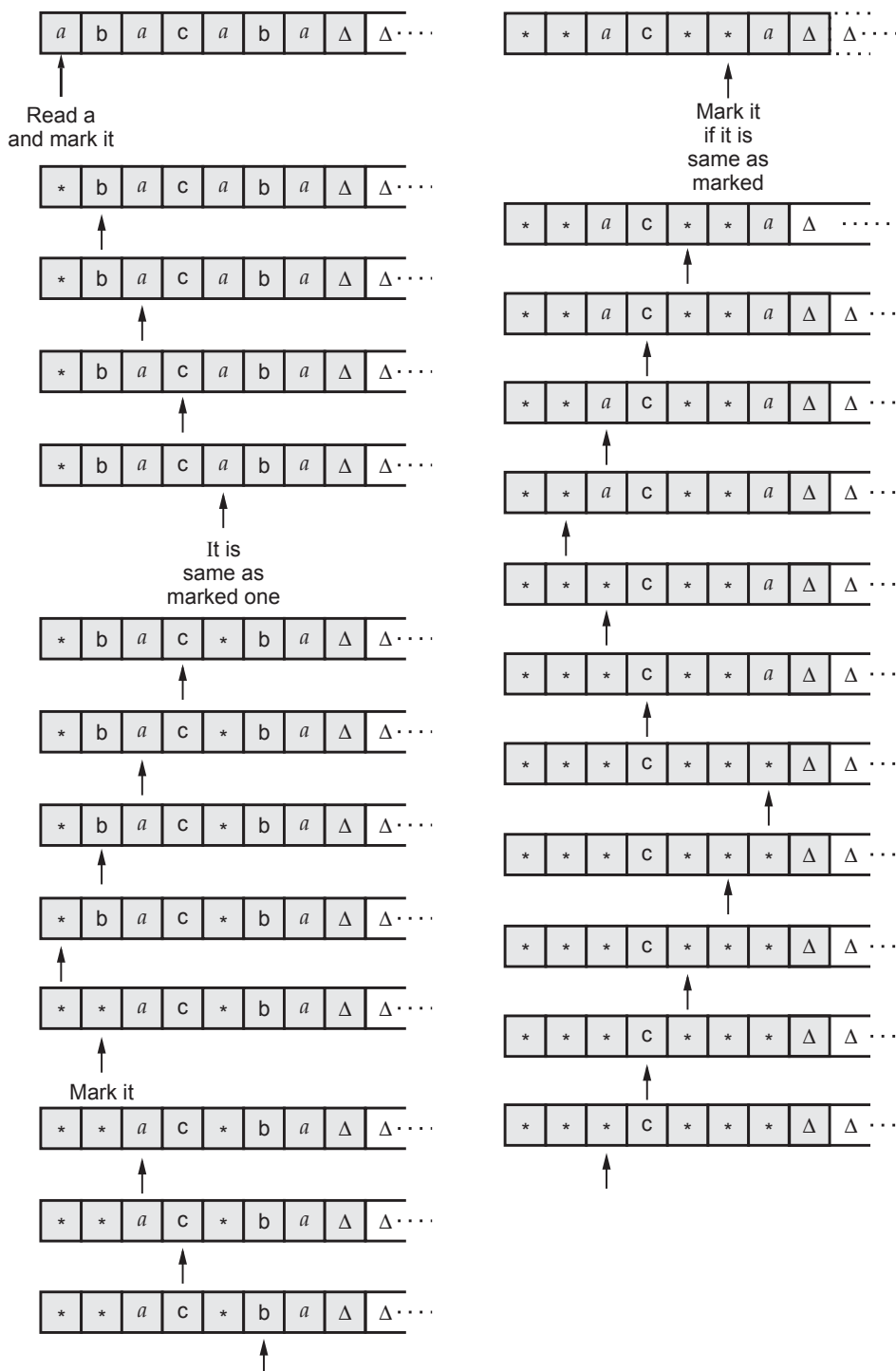


Fig. 5.4.4

Now machine goes to accept state. Thus in this TM we are scanning each symbol and trying to recognize the string.

5.4.4 Subroutine

In the high level languages use of subroutines built the modularity in the program development process. The same type of concept can be introduced in construction of TM. We can write the subroutines as a turing machine. Let us see how it works with the help of some example.

Example 5.4.3 Construct a TM for the subroutine $f(a, b) = a * b$

where a and b are unary numbers.

SPPU : May-19, End Sem, Marks 8

Solution : The unary number are represented by 1's. That if $a = 2$ and $b = 3$ then for this subroutine we will perform a copy operation. That is marking first 1 of a and copying it after Δ at b number of times, similarly marking second 1 of a and copying it at the rightmost end (i.e. after previously copied 1's) for b number of times. This makes the multiplication function complete.

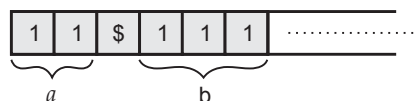


Fig. 5.4.5

			1	1	\$	1	1	1	Δ	
			$\uparrow$							
			X	1	\$	1	1	1	Δ	Moving to right upto \$
				$\uparrow$						
			X	1	\$	1	1	1	Δ	Copy this 1 after Δ by marking it as Y.
						$\uparrow$				
	X	1	\$	Y	1	1	Δ	1	Δ	Now move to left upto Y
								$\uparrow$		
	X	1	\$	Y	Y	1	1	Δ	1	Δ
	X	1	\$	Y	Y	1	1	Δ	1	Δ
						$\uparrow$				
	X	1	\$	Y	Y	1	1	Δ	1	Δ
						$\uparrow$				Mark this 1 as Y and copy this 1 at the rightmost end.
	X	1	\$	Y	Y	Y	1	1	1	Δ
										Now move to left upto Y
	X	1	\$	Y	Y	Y	Δ	1	1	1
	X	1	\$	Y	Y	Y	Δ	1	1	1
						$\uparrow$				

The head is pointing to Y next Y is Δ that means all the 1's are over we will convert these Y to 1's and repeat the same procedure for the 1 which is left to \$.

X	1	\$	1	1	1	Δ	1	1	1		
X	X	\$	1	1	1	Δ	1	1	1		Convert this 1 to X and move to right upto after the \$ symbol.
			↑								
X	X	\$	1	1	1	Δ	1	1	1		Mark it Y and copy this 1 to rightmost end
			↑								
X	X	\$	Y	1	1	Δ	1	1	1	1	Move to left upto Y
										↑	
X	X	\$	Y	1	1	Δ	1	1	1	1	Convert this 1 to Y and move to right, copy this 1 to rightmost end
				↑							
X	X	\$	Y	Y	1	Δ	1	1	1	1	Move to left upto Y
										↑	

The above steps will be repeated and all the 1's between \$ and Δ are copied to the rightmost end. This will be like this

X X \$ Y Y Y Δ 1 1 1 1 1 1

Now we will move to left by converting all Y's and X's to 1's. Finally input tape will have

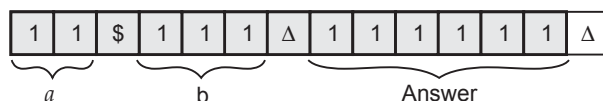


Fig. 5.4.6

We put these actions together and draw the transition graph and model out the turing machine.

The HALT state is a final state in the above TM.

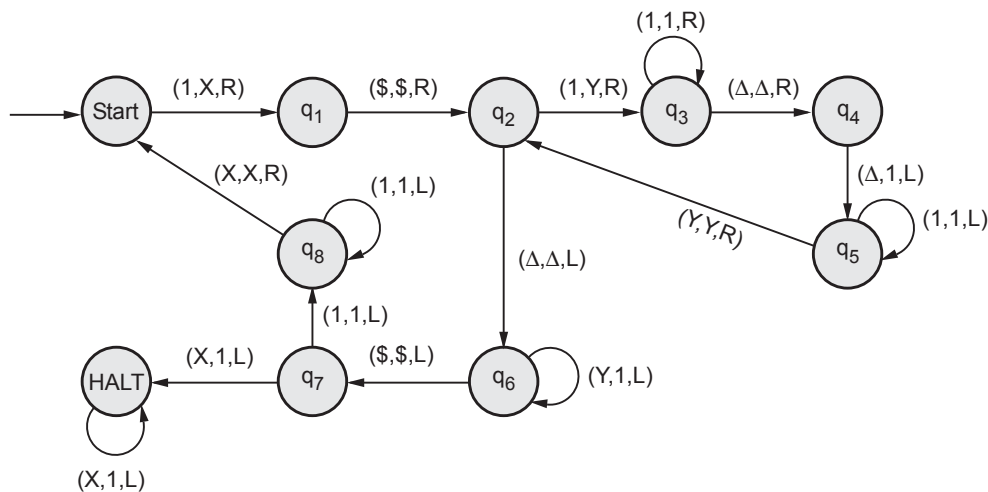


Fig. 5.4.7 Turing machine for multiplication subroutine

5.5 Language Acceptability by Turing Machines

SPPU : Dec.-06,07,09,11,13,14,17,18,19, May-06,11,13,14,16,18,19, Marks 10

The Turing machine accepts all the languages even though there are recursively enumerable. Recursive means repeating the same set of rules for any number of times. And enumerable means a list of elements. The TM also accepts the computable functions, such as addition, multiplication, subtraction, division, power function, square function, logarithmic function and many more. We will solve some examples based on languages and function for construction of Turing machine.

Example 5.5.1 Construct TM for language consisting of strings having any number of 0's and only even number of 1's over the input set $\Sigma = \{0, 1\}$.

Solution : The number of zero's can be any ! but the even number of 1's are to be allowed.

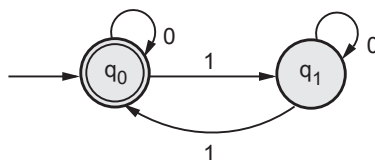


Fig. 5.5.1 FSM for even number of 1's

The same idea can be used to draw TM. We always assume that at the end of input Δ is placed.

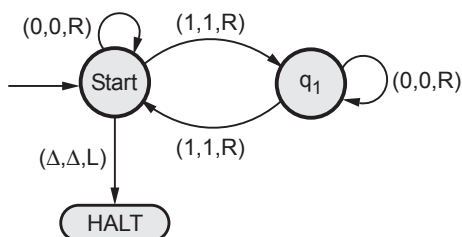
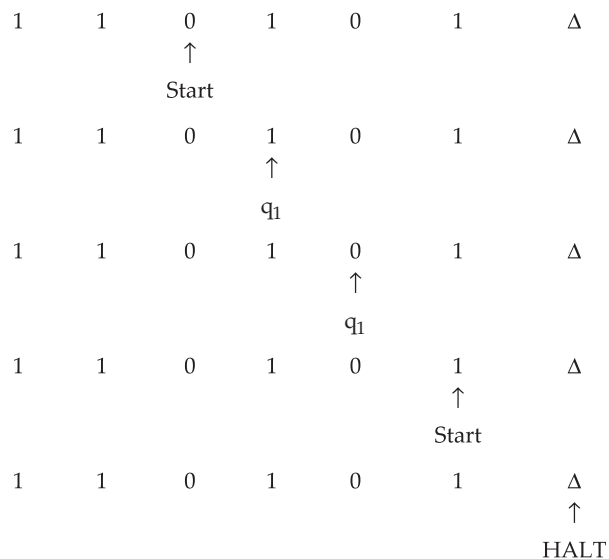


Fig. 5.5.2

Let us simulate the above TM for the input 1 1 0 1 0 1 which has even number of 1's. We assume that this input is placed on a input tape.

input	1	1	0	1	0	1	Δ
head	↑						
state	q_1						
	1	1	0	1	0	1	Δ
		↑					
		Start					



Thus this input is accepted by TM.

Example 5.5.2 Design a turing machine which recognizes the input language having a substring as 101 and replaces every occurrence of 101 by 110.

Solution : Replacement of any symbol by some another one means after reading of that specific symbol we should print the replacement symbol. In this case 101 has to be replaced by 110. The TM has to be constructed considering 101 as a substring and leaving 110 substring after complete scan of the input.

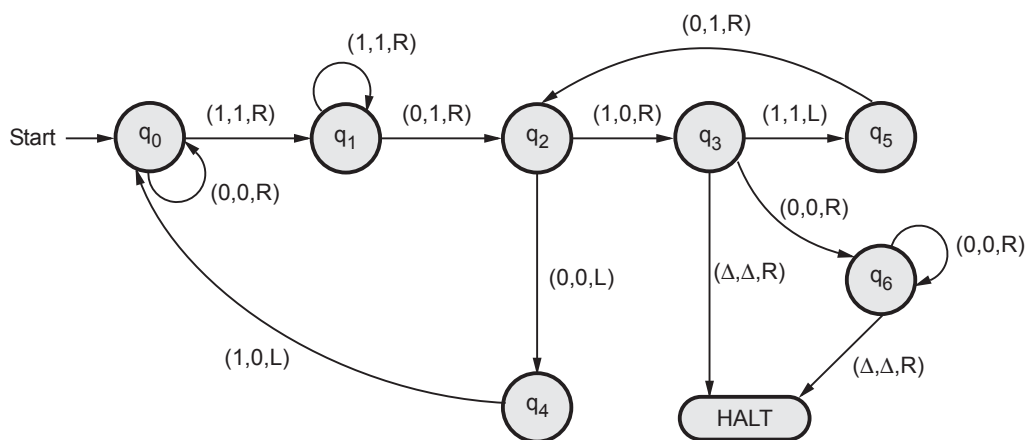


Fig. 5.5.3

Let us simulate it for the input.

1 0 0 1 0 1 0 Δ
 $\uparrow$

1 0 0 1 0 1 0 Δ
 $\uparrow$

1 0 0 1 0 1 0 Δ
 $\uparrow$

1 0 0 1 0 1 0 Δ
 $\uparrow$

1 0 0 1 1 1 0 Δ
 $\uparrow$

1 0 0 1 1 1 0 Δ
 $\uparrow$

1 0 0 1 1 0 0 Δ
 $\uparrow$

1 0 0 1 1 0 0 Δ
 $\uparrow$

read 1, print 1, goto q_1

goto R and next state is q_0

goto R, next state is q_0

goto R, next state q_1

print 1, goto R and next state is q_2

print 0, goto R and next state is q_3

print 0, goto R and next state is q_6

since we read current symbol as Δ we goto accept state i.e. HALT.

Example 5.5.3 Construct a TM for the language of even number of 1's and even number of 0's over $\Sigma = \{0, 1\}$.

SPPU : Dec.-17, End sem, Marks 8

Solution : For this type of problem we have drawn FSM, as follows -

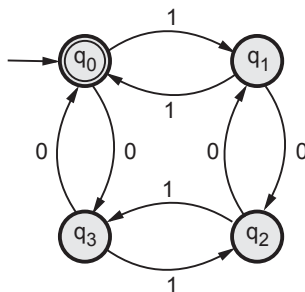


Fig. 5.5.4

And now it will be very much easy for us to convert this FSM to TM.

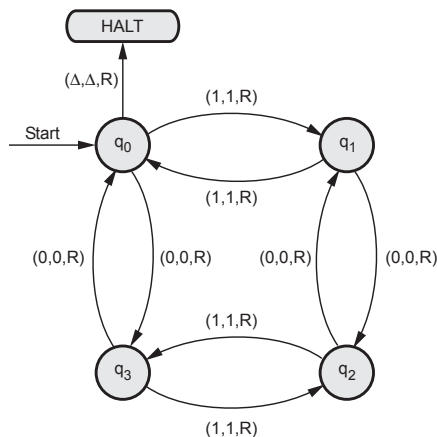


Fig. 5.5.5

Note that for any FSM we can construct turing machine. Thus turing machine more powerful than TM.

Example 5.5.4 Construct TM for the language $L = \{a^n b^n\}$ where $n \geq 1$.

SPPU : May-13, May-18, End Sem, Marks 8

Solution : The simple logic which we will apply is read out each a mark it by A and then move ahead along the input tape and find out the b convert it to B. Repeat this process for all a 's and b 's. If there is some kind of inequality in number of a 's and b 's the TM will not end up in HALT state. Let us formulize the logic for $a^n b^n$ and then construct it

$a \ a \ b \ b \ \Delta$	Convert it A and move right in search of b.
↑	
$A \ a \ b \ b \ \Delta$	Skip it, move ahead
↑	
$A \ a \ b \ b \ \Delta$	Convert it to B and move left till A
↑	
$A \ a \ B \ b \ \Delta$	Move left
↑	
$A \ a \ B \ b \ \Delta$	Move right
↑	
$A \ a \ B \ b \ \Delta$	Convert a to A and move right in search of b
↑	
$A \ A \ B \ b \ \Delta$	Move right
↑	
$A \ A \ B \ b \ \Delta$	Convert b to B and move left
↑	
$A \ A \ B \ B \ \Delta$	Move left
↑	
$A \ A \ B \ B \ \Delta$	Now immediately before B is A that means all the a 's are marked by A. So we will move right to ensure that no b is present
↑	
$A \ A \ B \ B \ \Delta$	Move right
↑	
$A \ A \ B \ B \ \Delta$	Move right
↑	
$A \ A \ B \ B \ \Delta$	HALT
↑	

Thus equality between a 's and b 's can be checked by moving tape head to and fro. Let us try to design a TM using above logic.

Let us design the transition table for the same.

	<i>a</i>	<i>b</i>	<i>A</i>	<i>B</i>	Δ
q_0	(q_1, A, R)	–	–	(q_3, B, R)	–
q_1	(q_1, a, R)	(q_2, B, L)	–	(q_1, B, R)	–
q_2	(q_2, a, L)	–	(q_0, A, R)	(q_2, B, L)	–
q_3	–	–	–	(q_3, B, R)	(HALT, Δ, L)
HALT	–	–	–	–	–

Table 5.5.1 Transition table for example 5.5.4

Thus TM does not lead to HALT state for such a invalid input.

Example 5.5.5 Construct a TM for $L = \{a^n b^n c^n \mid n \geq 1\}$.

Solution : This problem cannot be solved even by PDA. We have solved this type of problem by two stack PDA.

The simulation for *a a b b c c* can be shown as below.



Fig. 5.5.7

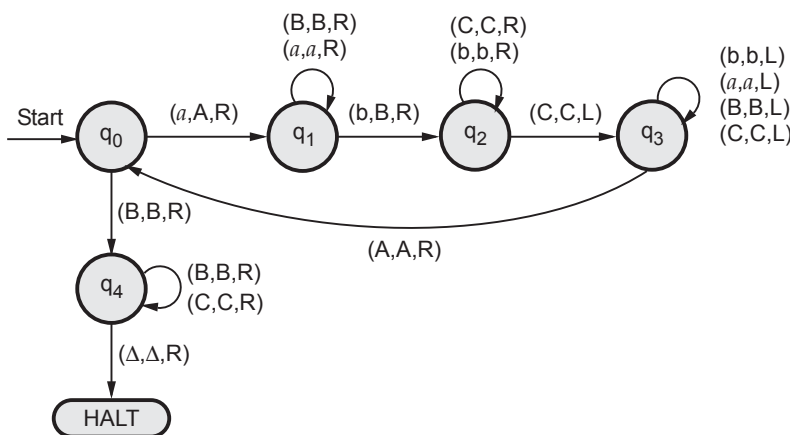


Fig. 5.5.8

$a a b b c c \Delta$ ↑	Convert a to A , move right the transition is from q_0 to q_1
$A a b b c c \Delta$ ↑	Move right upto b
$A a b b c c \Delta$ ↑	Convert b to B , move right, transition from q_1 to q_2
$A a B b c c \Delta$ ↑	Move right upto c
$A a B b c c \Delta$ ↑	Convert c to C , move right, transition from q_2 to q_3
$A a B B C c \Delta$ ↑	Move left upto C
$A a B B C c \Delta$ ↑	Move left till A , skipping a, b, B, C , see state q_3 then move one step right
$A a B B C c \Delta$ ↑	Convert a to A , move right
$A A B B C c \Delta$ ↑	Move right
$A A B b C c \Delta$ ↑	Convert b to B and move right, transition from q_1 to q_2
$A A B B C c \Delta$ ↑	Move right
$A A B B C c \Delta$ ↑	Convert c to C , move left till A , refer q_3 state
$A A B B C C \Delta$ ↑	Move right, keep B as it is refer transition from q_0 to q_4
$A A B B C C \Delta$ ↑	Go on moving right by ignoring B, C refer q_4 state
$A A B B C C \Delta$ ↑	Since Δ is read TM will reach to HALT state
$A A B B C C \Delta$ ↑	

Thus TM can very effectively recognize $a^n b^n c^n$ and more powerful than PDA.

Example 5.5.6 Construct a TM machine for checking the palindrome of the string of even length.

SPPU : May-11, Marks 8

* * * b b Δ ↑	Move to left till *
* * * b b Δ ↑	Move right
* * * b b Δ ↑	Replace it by *
* * * * b Δ ↑	Move right upto Δ
* * * * b Δ ↑	Move left to see if it is b, if so then replace it by Δ
* * * * Δ ↑	Move left till *
* * * * Δ ↑	Move right and check whether it is Δ , if so
* * * * Δ ↑	Goto HALT state

Let us draw TM for it,

If you observe the turing machine drawn for palindrome, you will find it very logically constructed. Even there is no need to insert any special symbol in between to separate out two halves.

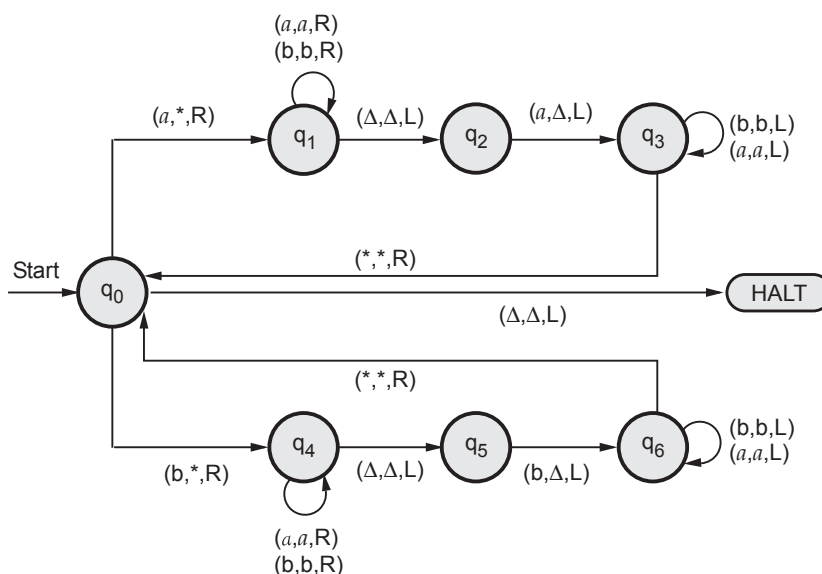


Fig. 5.5.9

Example 5.5.7 Construct a TM for checking the palindrome of a string of odd palindrome for $\Sigma = \{0, 1\}$.

SPPU : May-14, Marks 7, Dec.-19, End Sem, Marks 8

Solution : The TM will be very much similar to previous one but with a slight difference.

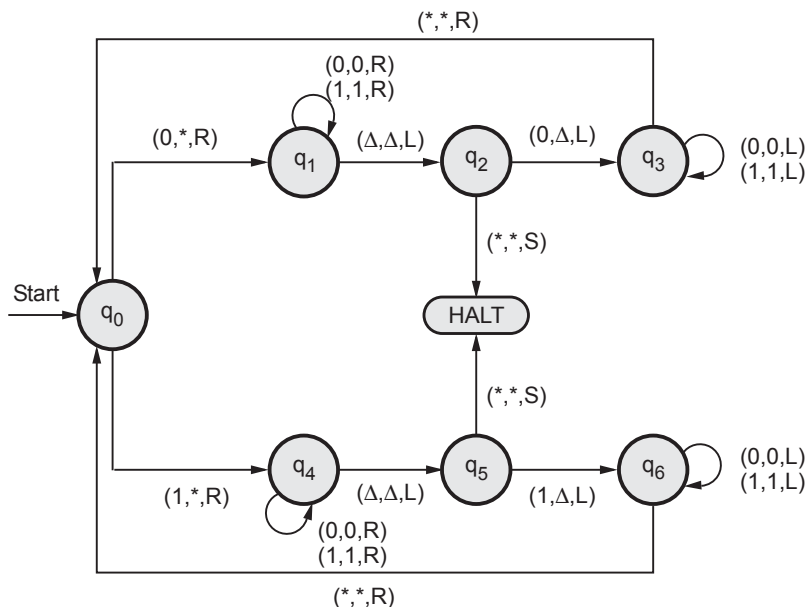


Fig. 5.5.10

For example : If the string is

0 0 1 0 0 Δ	Move right by making 0 to *
↑	
* 0 1 0 0 Δ	Move right upto Δ
↑	
* 0 1 0 0 Δ	Move left, replace 0 by Δ
↑	
* 0 1 0 Δ Δ	Move left
↑	
* 0 1 0 Δ	Move left upto *
↑	
* 0 1 0 Δ	Move right convert 0 to * and then move right
↑	
* * 1 0 Δ	Move right upto Δ
↑	

Let us assume that on input tape the input is -

1 1 1 \$ 1 1 1 Δ ↑	Move right till \$
1 1 1 \$ 1 1 1 Δ ↑	Move right
1 1 1 \$ 1 1 1 Δ ↑	If current symbol is 1 move left and convert \$ to 1
1 1 1 1 1 1 1 Δ ↑	Move right and convert 1 to \$
1 1 1 1 \$ 1 1 Δ ↑	Move right
1 1 1 1 \$ 1 1 Δ ↑	If current symbol is 1 move left and convert \$ to 1 and move right
1 1 1 1 1 1 1 Δ ↑	Convert 1 to \$ and move right
1 1 1 1 1 \$ 1 Δ ↑	Moved right
1 1 1 1 1 \$ 1 Δ ↑	If current symbol is 1 move left convert \$ to 1
1 1 1 1 1 1 1 Δ ↑	Move right convert 1 to \$
1 1 1 1 1 1 \$ Δ ↑	Move right
1 1 1 1 1 1 \$ Δ ↑	Since it is end of input we will move left convert \$ to Δ and HALT
1 1 1 1 1 1 Δ	

Example 5.5.9 Construct a TM for a successor function for a given unary number i.e. $f(n) = n+1$.

Solution : Since the TM has to be constructed for unary number system we assume input set $\Sigma = \{1\}$. The logic is very much simple. We just go on moving towards rightmost end of input string. The end marker of the string Δ is replaced by 1.

For example, the input tape consists of 4 the successor function will give output as 5.

1 1 1 1 Δ	We move to extreme right by keeping all 1's as it is
↑	
1 1 1 1 Δ	Move right
↑	
1 1 1 1 Δ	Move right
↑	
1 1 1 1 Δ	Move right
↑	
1 1 1 1 Δ	Convert Δ to 1
↑	
1 1 1 1 1 Δ	HALT

The construction of TM will be,

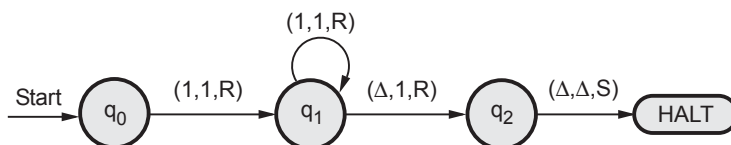


Fig. 5.5.13

The another method of representation of TM will be by transition table. It is always convenient to build transition table from the graph.

Input \ States	1	Δ
q_0	$(q_1, 1, R)$	–
q_1	$(q_1, 1, R)$	$(q_2, 1, R)$
q_2	–	(HALT, Δ, S)
HALT	–	–

In the transition table the triple is of (next state, output, direction).

Example 5.5.10 Construct TM for the addition function for the unary number system.

SPPU : Dec.-17, 19, End Sem,Marks 6

Solution : The unary number is made up of only one character i.e. The number 5 can be written in unary number system as 11111. In this TM we are going to perform addition of two unary numbers.

For example $2 + 3$
 i.e. $11 + 111$
 $= 11111$

If you observe this process of addition, you will find the resemblance with string concatenation function.

What we are trying to do is, we simply replace $+$ by 1 and move ahead right for searching end of the string we will convert last 1 to Δ .

The input is $3 + 2$.

1 1 1 + 1 1 Δ	Move right upto $+$ sign
↑	
1 1 1 + 1 1 Δ	Convert $+$ to 1 and move right
↑	
1 1 1 1 1 1 Δ	Move right
↑	
1 1 1 1 1 1 Δ	Move right
↑	
1 1 1 1 1 1 Δ	Now Δ has encountered so just move left
↑	
1 1 1 1 1 1 Δ	Convert 1 to Δ
↑	
1 1 1 1 1 $\Delta \Delta$	Thus the tape now consists of the addition of two unary numbers

The TM will look like this

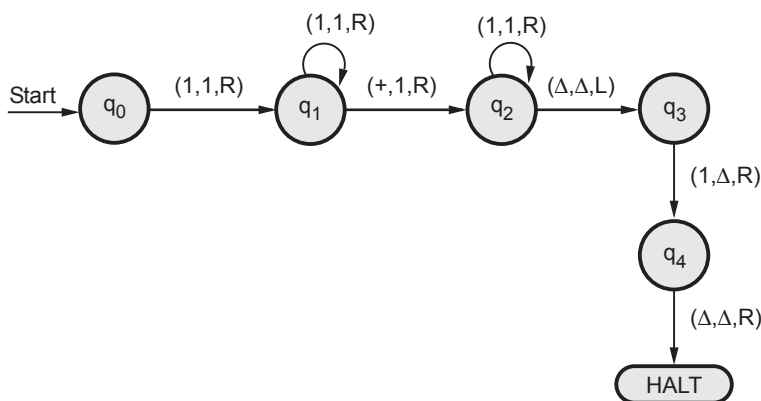


Fig. 5.5.14

Here we are implementing the function of $f(a + b) = c$. We assume a and b both are nonzero elements.

Example 5.5.11 Construct TM for performing subtraction of two unary numbers $f(a-b)=c$ where a is always greater than b .

Solution : Here we have certain assumption as first number is greater than second one. Let us assume that $a = 3$, $b = 2$, so the input tape will be,

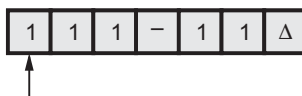


Fig. 5.5.15

We will move right to $-$ symbol as perform reduction of number of 1's from first number. Let us look at the simulation for understanding the logic.

1 1 1 - 1 1 Δ ↑	Move right upto '-'
1 1 1 - 1 1 Δ ↑	Move right and convert 1 to *
1 1 1 - * 1 Δ ↑	Now move left
1 1 1 - * 1 Δ ↑	Move left
1 1 1 - * 1 Δ ↑	Convert 1 to *
1 1 * - * 1 Δ ↑	Move right
1 1 * - * 1 Δ ↑	Move right till 1
1 1 * - * 1 Δ ↑	Convert 1 to * and move left
1 1 * - * * Δ ↑	Move left till 1
1 1 * - * * Δ ↑	Convert 1 to * and move right
1 * * - * * Δ ↑	Move right till Δ
1 * * - * * Δ ↑	HALT

Thus we get 1 on the input tape as answer for $f(3 - 2)$.

Let us draw TM for it.

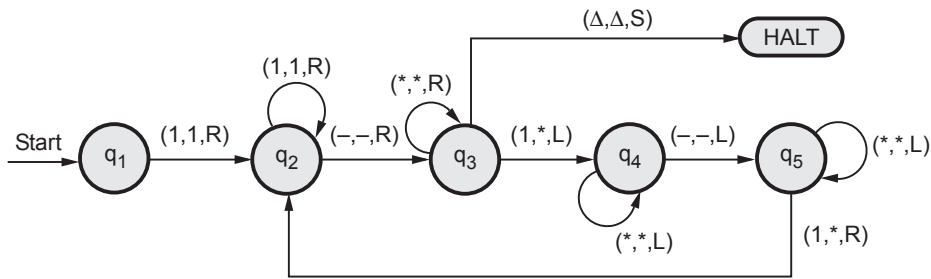


Fig. 5.5.16

Note that we have enforced a condition that first number is greater than the second. Even without such condition also the TM can be drawn by introducing some extra logic in existing TM. This part is left as exercise for the reader.

Example 5.5.12 Construct turing machine that recognizes the language

- 1) $L = \{0^n 1^m : n, m \geq 0\}$
- 2) $L = \{x \in \{0,1\}^* \mid x \text{ ends in } 00\}$

SPPU : May-06, Marks 10

Solution :

i) Let, $\{L = 0^n 1^m \mid n, m \geq 0\}$

That means any number of 0's and 1's are allowed in a string but all the 0's must appear before all the 1's. Hence the TM can be,

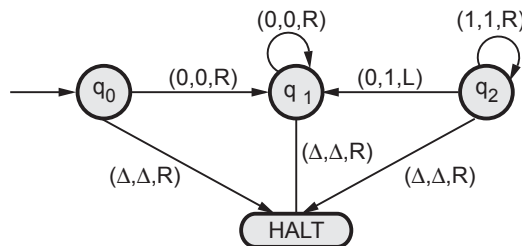


Fig. 5.5.17

ii) Let, $L = \{x \in \{0,1\}^* \mid x \text{ end in } 00\}$ be the given language. We will first draw FA for this language.

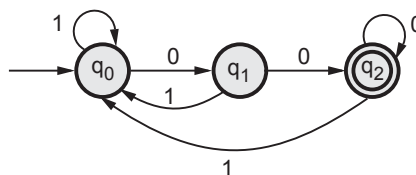


Fig. 5.5.18

The TM can then be as follows,

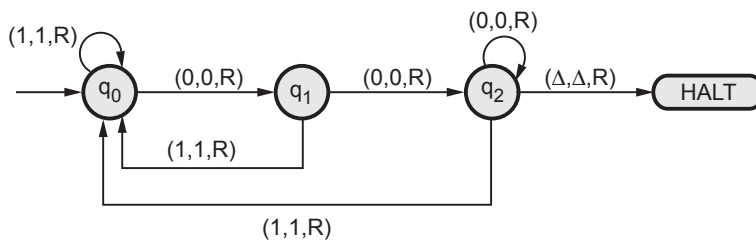


Fig. 5.5.19

Example 5.5.13 Design a turing machine to replace string 110 by 101 in binary input string.

SPPU : Dec.-06, Marks 8, Dec.-14, End Sem., Marks 9

Solution : The TM can then be,

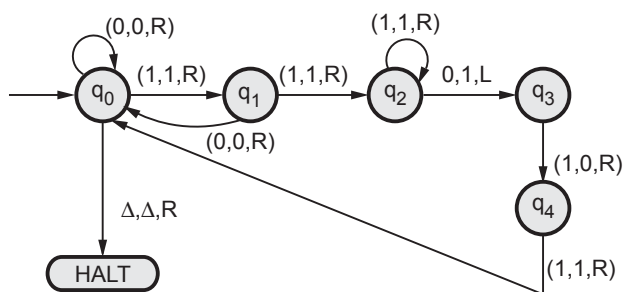


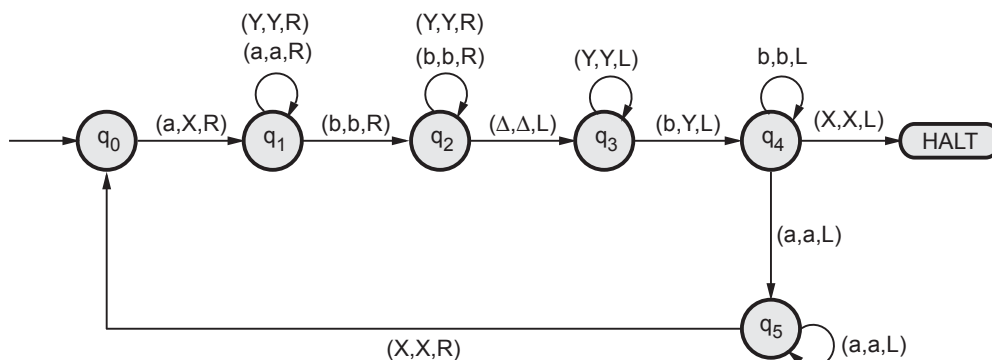
Fig. 5.5.20

Example 5.5.14 Draw a transitions table for turing machine accepting each of the following languages.

- a) $\{a^i b^j / i < j\}$ b) The language of balanced strings of parentheses.

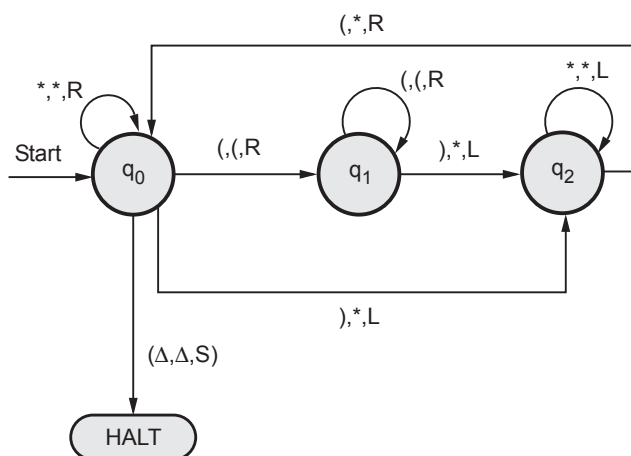
SPPU : Dec.-07, 09, May-09, Marks 8, Dec.-14, End Sem., Marks 9

Solution : a) The required TM can be drawn as follows -



The transition table can be

Input \ State	a	b	X	Y	Δ
q_0	(q_1, X, R)	–	–	–	–
q_1	(q_1, a, R)	(q_2, b, R)	–	–	–
q_2	–	(q_2, b, R)	–	(q_2, Y, R)	(q_3, Δ, L)
q_3	–	(q_4, Y, L)	–	(q_3, Y, L)	–
q_4	(q_5, a, L)	(q_4, b, L)	(HALT, X, L)	–	–
q_5	(q_5, a, L)	–	(q_0, X, R)	–	–



The transition table for this TM will be

State	(	)	*	Δ
q_0	$(q_1, (, R)$	$(q_2, *, L)$	$(q_0, *, R)$	(HALT, Δ, S)
q_1	$(q_1, (, R)$	$(q_2, *, L)$	–	–
q_2	$(q_2, *, R)$	–	$(q_2, *, L)$	–
HALT	–	–	–	–

Example 5.5.15 Design a turing machine M to recognize the language $\{1^n 2^n 3^n \mid n \geq 1\}$.

SPPU : Dec.-11, Marks 8; Dec.-13, Marks 6

Solution :

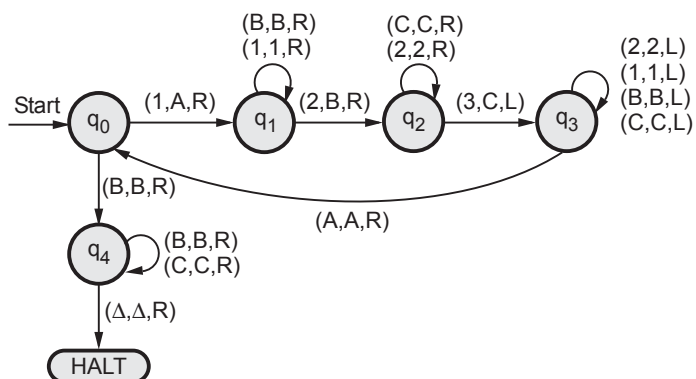


Fig. 5.5.21

Example 5.5.16 Design a TM that replaces every occurrence of abb by baa.

SPPU : May-16, Marks 6

Solution :

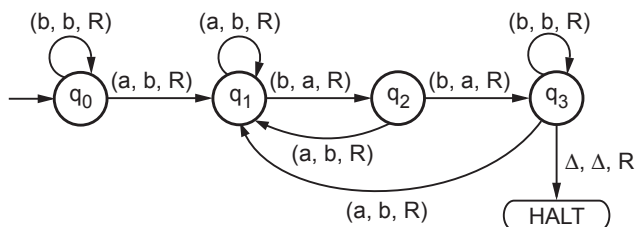


Fig. 5.5.22

Example 5.5.17 Construct TM for 1's complement of binary number.

SPPU : May-18, End Sem, Marks 6

Solution :

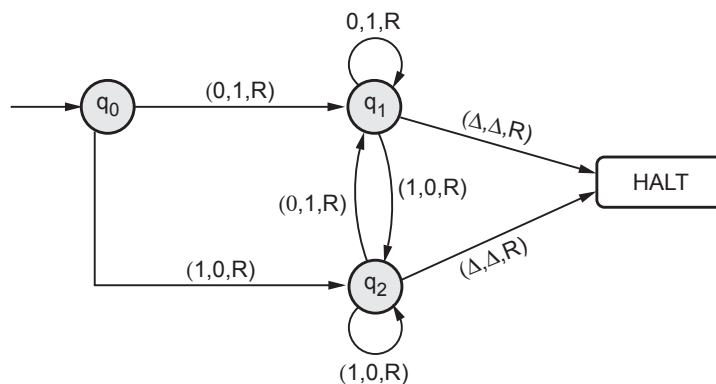


Fig. 5.5.23

Example 5.5.18 Design a turing machine to accept the language $L = \{w/w \in (0 + 1)^*\}$ containing the substring 001.

SPPU : May-18, End Sem, Marks 8

Solution :

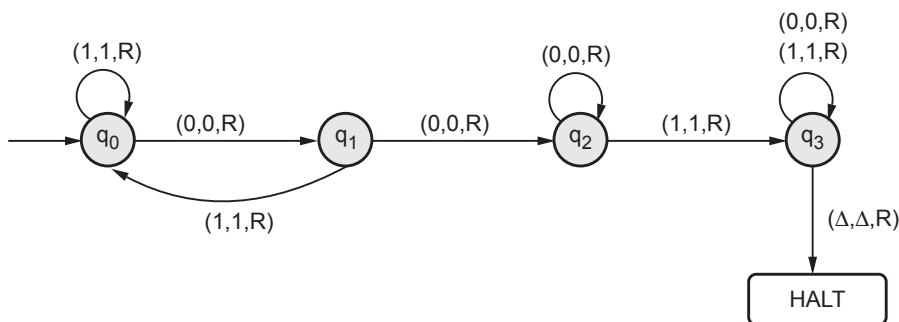
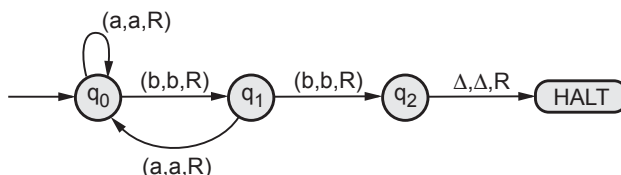


Fig. 5.5.24

Example 5.5.19 Construct a turing machine for $R = (a + b)^*bb$.

SPPU : Dec.-18, End Sem, Marks 6

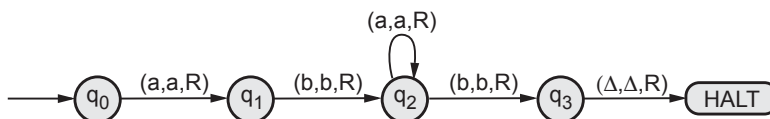
Solution :



Example 5.5.20 Construct a turing machine for $R = (aba^*b)$.

SPPU : Dec.-18, End Sem, Marks 6

Solution :



Example 5.5.21 Design TM for the language $L = \{0^{2n}\}$ over $\Sigma = \{0,1\}$.

SPPU : May-19, End Sem, Marks 8

Solution : We will design finite automata for

$$L = \{0^{2n}\} \text{ over } \{0, 1\}$$

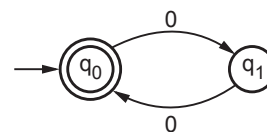


Fig. 5.5.25

From this FA we can design TM as follows -

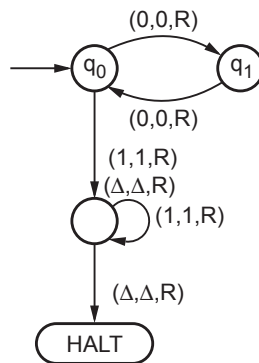


Fig. 5.5.26

Example 5.5.22 Design TM to accept the set L of all strings formed with 0 and 1 and having substring '000'.

SPPU : May-19, End Sem, Marks 8

Solution : The FA can be designed as,

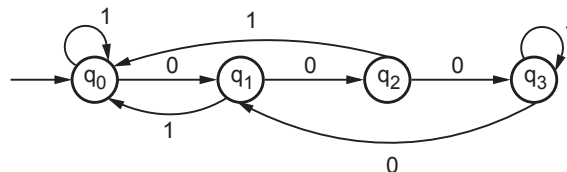


Fig. 5.5.27

From this FA we can design TM as,

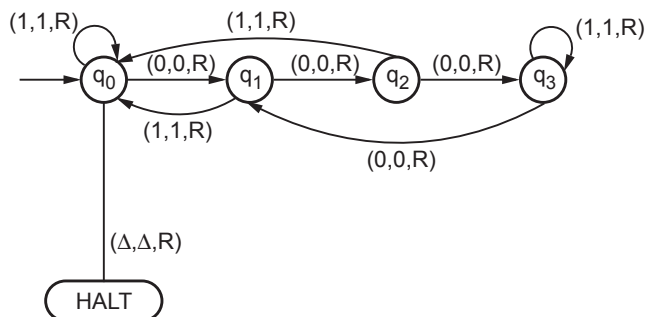


Fig. 5.5.28

Example 5.5.23 Construct a Turing machine for the language $L = \{a^n bc^n \mid n \geq 1\}$.

SPPU : Dec.-19, End Sem, Marks 6

Solution :

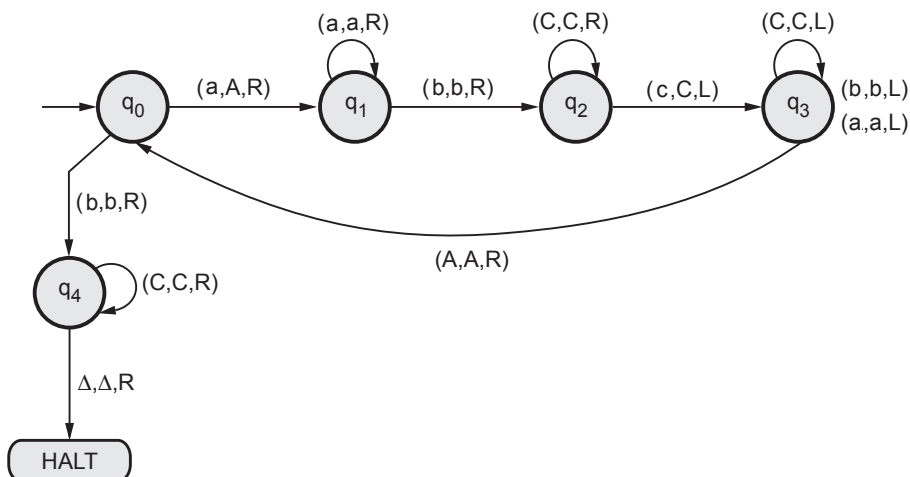


Fig. 5.5.29

5.6 Computing Function with Turing Machine

SPPU : Dec.-05,07,08,09,10,15,17,18,19, May-06,07,10,12,14,15,16, Marks 12

- The TM can perform string operations, arithmetic computations, recognizing languages either regular or nonregular languages.
- In this section we are going to see more power of TM by allowing the two way infinite tape as shown in Fig. 5.6.1.
- The tape head as usual can move in forward and backward direction.



Fig. 5.6.1 Two way infinite tape

Theorem : L is recognized by a turing machine with a two way infinite tape if and only if it is recognized by a TM with one way infinite tape.

Proof : As theorem states, if any language is recognized by a TM with one way infinite tape then it should also be recognized by a TM with two way infinite tape.

Let M_1 be a TM with one way infinite tape and can be denoted by,

$$M_1 = (Q_1, \Sigma_1, \Gamma_1, \delta_1, q_1, B, F_1)$$

Similarly, M_2 be a TM with two way infinite tape and can be denoted by,

$$M_2 = (Q_2, \Sigma_2, \Gamma_2, \delta_2, S_1, B, F_2)$$

The input tapes are as shown by following figure.

In Fig. 5.6.2, the TM with one way infinite tape has a external symbol # placed in the very first cell from left side. This symbol is used as indicator for the left side termination.

If we want a language $L = \{q_0, q_1, q_2, q_3, q_4, q_5\}$ sequence then in the two way infinite tape the tape head is fixed at the rightmost symbol.

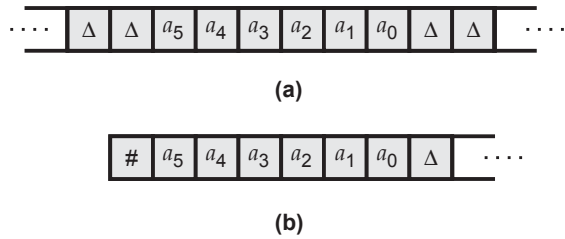


Fig. 5.6.2 (a) Two way infinite tape for M_2 TM
(b) One way infinite tape for M_1 TM

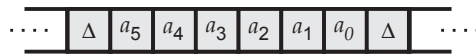


Fig. 5.6.3

The TM M_2 can be

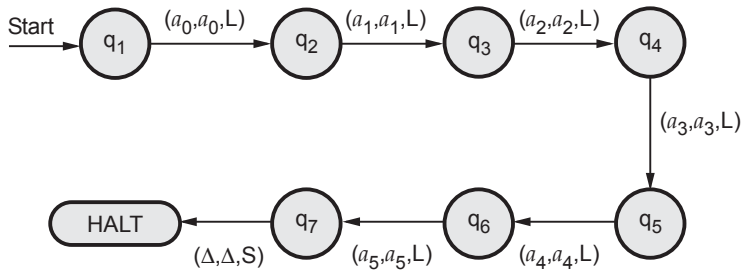


Fig. 5.6.4

$\delta(q_7, \Delta) \rightarrow (\text{HALT}, \Delta, S)$ leads to HALT state.

Similarly, with one way infinite tape the machine M_1 will be

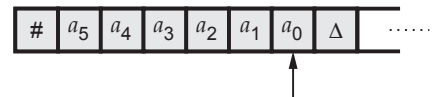


Fig. 5.6.5 (a)

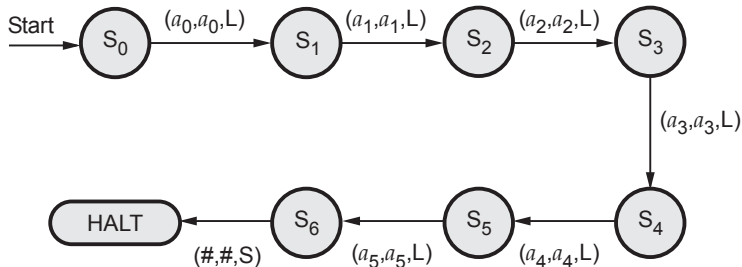


Fig. 5.6.5 (b)

Even we can make the TM with one way infinite tape as a multitrack tape to simulate it as a two way infinite tape. Let us now solve some interesting problems to see the working of two way infinite tape in the turing machine.

Example 5.6.1 Construct a TM for a language having equal number of a 's and b 's in it over the input set $\Sigma = \{a, b\}$. **SPPU : Dec.-08, Marks 6; May-12, Marks 8; May-14, Marks 7, Dec.-17,18, End Sem, Marks 8**

Solution : Here we will clearly take the advantage of infinite two ends of the input tape. The input can be placed as follows.

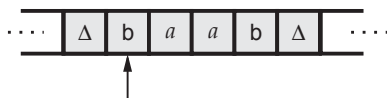


Fig. 5.6.6

We will have a simple logic as if we get a we will replace it by A and move right in search of b , if we get b we will replace it by B and move left. We will move to the leftmost end (i.e. Δ) and again repeat the same process of marking a by A and b by B . We may have string starting with a or b , so from initial state only we will have two separate paths shown for a and b . Remember we have to check both the ends, to check whether the string is completely scanned or not. Let us understand the logic first and then move on for TM.

$\Delta b a a b \Delta$	Convert b to B and move right in search of a
$\uparrow$	
$\Delta B a a b \Delta$	a has to be converted to A and move left
$\uparrow$	
$\Delta B A a b \Delta$	Skip B and move left
$\uparrow$	
$\Delta B A a b \Delta$	Keep Δ as it is and move right
$\uparrow$	
$\Delta B A a b \Delta$	Move right
$\uparrow$	
$\Delta B A a b \Delta$	Move right
$\uparrow$	
$\Delta B A a b \Delta$	Convert a to A and move right
$\uparrow$	
$\Delta B A A b \Delta$	Convert b to B move left
$\uparrow$	
$\Delta B A A B \Delta$	Move left
$\uparrow$	

$\Delta B A A B \Delta$	Move left
$\uparrow$	
$\Delta B A A B \Delta$	Move left
$\uparrow$	
$\Delta B A A B \Delta$	If Δ comes move to right by ignoring all A's and B's
$\uparrow$	
$\Delta B A A B \Delta$	Move right
$\uparrow$	
$\Delta B A A B \Delta$	Move right
$\uparrow$	
$\Delta B A A B \Delta$	Move right
$\uparrow$	
$\Delta B A A B \Delta$	Move right
$\uparrow$	
$\Delta B A A B \Delta$	Δ is reached, goto HALT state
$\uparrow$	

The TM can be

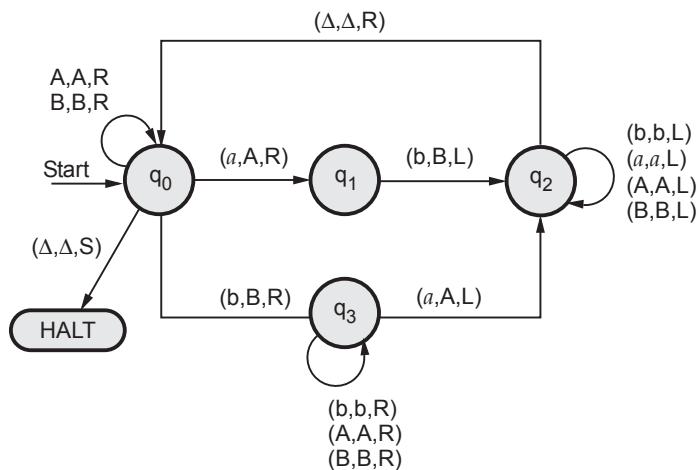


Fig. 5.6.7

Similarly if the string is

$\Delta a b a b \Delta$	Convert a to A move right
$\uparrow$	
$\Delta A b a b \Delta$	Convert b to B move left
$\uparrow$	

$\Delta A B a b \Delta$ ↑	Skip A and move left
$\Delta A B a b \Delta$ ↑	Move right, ignore all A's and B's
$\Delta A B a b \Delta$ ↑	Convert a to A, move right
$\Delta A B A b \Delta$ ↑	Convert b to B, move left
$\Delta A B A B \Delta$ ↑	Skip all A's and B's and move left
$\Delta A B A B \Delta$ ↑	Move right by skipping all A's and B's
$\Delta A B A B \Delta$ ↑	Since both the ends are checked and all the a 's and b 's are encountered as equal. So TM HALTs here.

Example 5.6.2 Construct TM for obtaining two's complement of a given binary number.

SPPU : Dec.-05, Marks 10; Dec.-10, Marks 8, Dec.-19, End Sem, Marks 6

Solution : The logic for computing two's complement is, we read the binary string from LSB to MSB. From LSB, we keep all the zero's as it is and move left till we do not get 1. After reading first 1 from LSB, we move left and thereafter we convert 0 to 1 and 1 to 0 and go on moving towards left. The process continues upto leftmost Δ .

For example : The binary number 0110 its two's complement can be computed as

$$\begin{array}{r}
 0110 \\
 \hline
 1001 \quad \text{1's complement} \\
 + \quad 1 \quad \text{Adding 1} \\
 \hline
 = 1010 \quad \text{Two's complement of 0110}
 \end{array}$$

By our idea also we will get the same result.

$\Delta 0110 \Delta$ ↑	Move to rightmost end upto Δ
$\Delta 0110 \Delta$ ↑	Move left
$\Delta 0110 \Delta$ ↑	It is 0, so keep it as it is and move left
$\Delta 0110 \Delta$ ↑	First 1 from LSB has encountered so keep it as it is and move left

$\Delta 0 1 1 0 \Delta$	Complement it to 0 and move left
↑	
$\Delta 0 0 1 0 \Delta$	Convert it to 1 and move left
↑	
$\Delta 1 0 1 0 \Delta$	Since Δ is reached, stop !
↑	

The TM could be

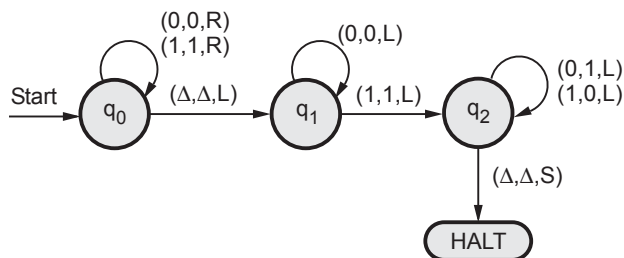


Fig. 5.6.8

Thus two way infinite tape brings significant power to turing machine.

Example 5.6.3 Construct TM for checking well formness of the parenthesis.

Solution : We can solve this problem with PDA as well as with TM also. The parenthesis should be well formed in case of expression solving. The logic which will apply will be very much similar to the logic for finding equal number of *a*'s and equal number of *b*'s. We will start moving towards right, when we get any ')' we will mark it as * and move left in search of '('. Mark it as *. Thus we will repeat this process for the complete input string.

For example

$\Delta () () \Delta$	Move right in search of ')', skipping '('
↑	
$\Delta () () \Delta$	Mark it as * and move left, in search of '('
↑	
$\Delta (* () \Delta$	Now '(' has encountered mark it as *. Move left in search of '(', mark it *
↑	
$\Delta * * () \Delta$	Move right, skip *
↑	
$\Delta * * () \Delta$	Move right
↑	
$\Delta * * () \Delta$	Convert to * and move left
↑	

$\Delta * * (* \Delta$
 $\uparrow$
 Convert (to *
 $\Delta * * * \Delta$
 $\uparrow$
 Move right till Δ
 $\Delta * * * \Delta$
 $\uparrow$
 HALT

The TM will look like this -

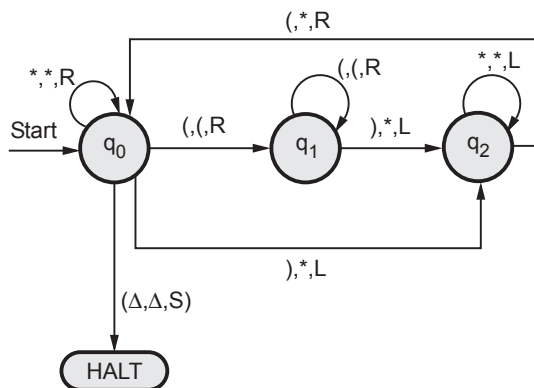


Fig. 5.6.9

The transition table for this TM will be

State	(	)	*	Δ
q ₀	(q ₁ , (, R)	(q ₂ , *, L)	(q ₀ , *, R)	(HALT, Δ, S)
q ₁	(q ₁ , (, R)	(q ₂ , *, L)	—	—
q ₂	(q ₂ , *, R)	—	(q ₂ , *, L)	—
HALT	—	—	—	—

Example 5.6.4 Construct TM for reversing a binary string on the input tape.

SPPU : Dec.-09, May-10, Marks 10

Solution : Suppose on the input tape some binary string is placed, then we will read each character from left to right and copy it at the end of the tape.

For example

$\dots \Delta 1 0 1 0 0 1 \Delta \Delta \Delta \dots$
 $\uparrow$
 We will move upto the end of the input string

Δ 1 0 1 0 0 1 Δ Δ ...
 ↑

Move left

Δ 1 0 1 0 0 1 Δ Δ ...
 ↑

Mark it * and copy it after Δ

Δ 1 0 1 0 0 * Δ 1
 ↑

Move left upto *

Δ 1 0 1 0 0 * Δ 1 Δ
 ↑

Move left, convert 0 to * and place it, at the end

Δ 1 0 1 0 * * Δ 1 0 Δ
 ↑

Move left upto *

Δ 1 0 1 0 * * Δ 1 0 Δ
 ↑

Move left, convert 0 to * and place it at the end

Δ 1 0 1 * * * Δ 1 0 0 Δ
 ↑

Move left upto *

Δ 1 0 1 * * * Δ 1 0 0 Δ
 ↑

Move left, convert 1 to * and place it at the end

Δ 1 0 * * * * Δ 1 0 0 1 Δ
 ↑

Move left upto *

Δ 1 0 * * * * Δ 1 0 0 1 Δ
 ↑

Move left, convert 0 to * and place it at the end

Δ 1 * * * * * Δ 1 0 0 1 0 Δ
 ↑

Move left skipping *

Δ 1 * * * * * Δ 1 0 0 1 0 Δ
 ↑

Copy 1 at the end of tape by converting 1 to *

Δ * * * * * Δ 1 0 0 1 0 1 Δ
 ↑

Move left skipping all *

Δ * * * * * Δ 1 0 0 1 0 1 Δ
 ↑

Left to * is a Δ character so we goto HALT state

The TM will be,

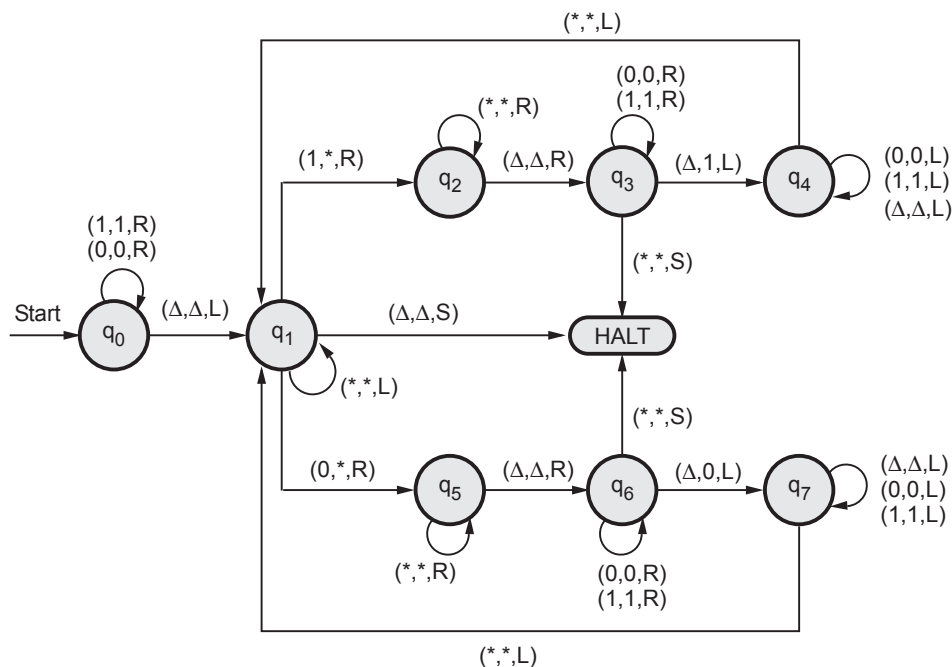


Fig. 5.6.10

The transition table for the above TM will be,

	0	1	*	Δ
q ₀	(q ₀ , 0, R)	(q ₀ , 1, R)	–	(q ₂ , Δ , L)
q ₁	(q ₅ , *, R)	(q ₂ , *, R)	(q ₁ , *, L)	(HALT, Δ , S)
q ₂	–	–	(q ₂ , *, R)	(q ₃ , Δ , R)
q ₃	(q ₃ , 0, R)	(q ₃ , 1, R)	–	(q ₄ , 1, L)
q ₄	(q ₄ , 0, L)	(q ₄ , 1, L)	(q ₁ , *, L)	(q ₄ , Δ , L)
q ₅	–	–	(q ₅ , *, R)	(q ₆ , Δ , R)
q ₆	(q ₆ , 0, R)	(q ₆ , 1, R)	–	(q ₇ , 0, L)
q ₇	(q ₇ , 0, L)	(q ₇ , 0, L)	(q ₁ , *, L)	(q ₇ , Δ , L)
HALT	–	–	–	–

Thus TM reverses the input string.

Example 5.6.5 Construct TM for copying the input unary string on the tape.

SPPU : May-14, Marks 8

Solution : The turing machine is able to perform copy operation also. In this TM will read each symbol mark it as A (to keep track of visited symbols) and copy it at the right end after Δ .

Finally after copying the complete string we will move left and reconvert 'A' to 1 again.

The TM will be as follows.

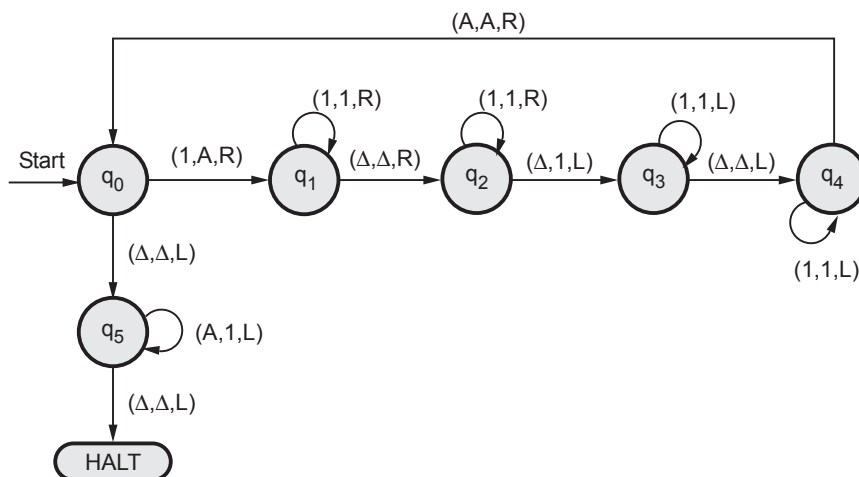


Fig. 5.6.11

Let us simulate it.

.... Δ 1 1 1 1 Δ ...

↑

Mark 1 as A and move right refer start state q_0

Δ A 1 1 1 Δ

↑

Keep 1 as it is and move right, see q_1 state

Δ A 1 1 1 Δ

↑

Move right

Δ A 1 1 1 Δ

↑

Move right

Δ A 1 1 1 Δ Δ

↑

Move right simply transition from q_1 to q_2

Δ A 1 1 1 Δ 1

↑

Converted Δ to 1 and move left

Δ A 1 1 1 Δ 1

↑

Move left

$\Delta A 1 1 1 \Delta 1$ $\uparrow$	Move left by keeping 1's as it is till 'A'
$\Delta A 1 1 1 \Delta 1$ $\uparrow$	Move right
$\Delta A 1 1 1 \Delta 1$ $\uparrow$	Convert 1 to A and move right upto Δ
$\Delta A A 1 1 \Delta 1 \Delta$ $\uparrow$	Move right even after 1
$\Delta A A 1 1 \Delta 1 \Delta$ $\uparrow$	Convert Δ to 1 and move left upto A
$\Delta A A 1 1 \Delta 1 1$ $\uparrow$	Move right convert 1 to A
$\Delta A A A 1 \Delta 1 1 \Delta$ $\uparrow$	Convert Δ to 1
$\Delta A A A 1 \Delta 1 1 1$ $\uparrow$	Move left upto A
$\Delta A A A 1 \Delta 1 1 1$ $\uparrow$	Move right convert 1 to A and copy it at the end
$\Delta A A A A \Delta 1 1 1 1$ $\uparrow$	Move left upto Δ
$\Delta A A A A \Delta 1 1 1 1$ $\uparrow$	Move left and convert A to 1 upto Δ
$\Delta A A A 1 \Delta 1 1 1 1$ $\uparrow$	
$\Delta 1 1 1 1 \Delta 1 1 1 1$ $\uparrow$	Here TM HALT

Example 5.6.6 Construct TM for performing logarithmic operation of given binary number i.e. compute $\log(n)$ function.

Solution : For obtaining logarithmic value of the binary number we will enforce a condition as, that binary number has to be in the power of 2 form.

That means 1, 10, 100, 1000, 10000, are valid inputs for our TM.

$$\log(1) = 0$$

$$\log(10) = \log(2) = 1$$

$$\log(100) = \log(4) = 2$$

Observe the above examples carefully and you will find that the number of zeros following 1 is the answer for logarithmic value.

For example

$$\begin{aligned}\log(100) &= \text{Number of zeros after 1 and 2 are even answer} \\ &= 2\end{aligned}$$

The TM will be as follows -

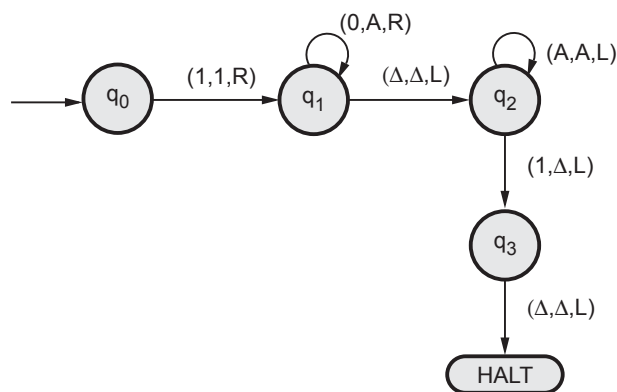


Fig. 5.6.12

Let us take some input as $\log(8) = 3$.

Δ 1 0 0 0 Δ	Move right
↑	
Δ 1 0 0 0 Δ	Convert 0 to A and move right
↑	
Δ 1 A 0 0 Δ	Convert 0 to A and move right
↑	
Δ 1 A A 0 Δ	Convert 0 to A and move right
↑	
Δ 1 A A A Δ	Now move left
↑	
Δ 1 A A A Δ	Move left, ignoring A's upto 1
↑	
Δ 1 A A A Δ	Convert 1 to Δ
↑	
Δ Δ A A A Δ	Move left
↑	
Δ Δ A A A Δ	The answer is in the unary form and that is number of A's = 3
↑	

Example 5.6.7 Design TM to recognize an arbitrary string divisible by 4 from $\Sigma = \{0, 1, 2\}$.

SPPU : May-15, End Sem, Marks 10

Solution : As input string is $\Sigma = \{0, 1, 2\}$, any arbitrary string of input should be a ternary string and we have check whether such ternary input is divisible by 4 or not. For divisibility by 4 there are four case -

Remainder 0, Remainder 1, Remainder 2, Remainder 3

Now we will consider separate states for each of these cases -

q_0 - Remainder 0 state

q_1 - Remainder 1 state

q_2 - Remainder 2 state

q_3 - Remainder 3 state

HALT : is a final. When you get a valid string (string which is divisible by 4). The TM is using a triplet of (input, output, direction) during the transition from one state to another.

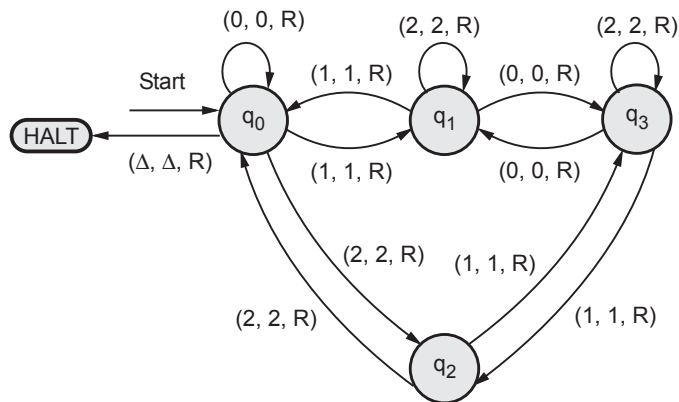


Fig. 5.6.13

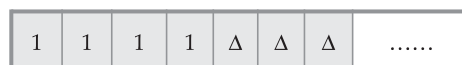


Input tape with sample input

For the sample input string 121 Δ we will reach to halt state finally as $(121)_3 = (16)_{10}$ which is divisible by 4.

Example 5.6.8 Construct TM for the function $f(x) = x + 3$.

Solution : We will represent number x using unary number. Hence the input set $\Sigma = \{1\}$. If we assume $x = 4$ then sample input tape can be represented as



Input tape with $x = 4$

Addition by constant 3 means we will simply convert last three Δ 's to 1. Hence the required TM will be -

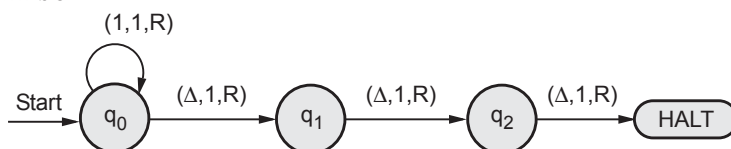


Fig. 5.6.14

The transition table can be as shown below.

Input \ State	1	Δ
q_0	$(1, q_0, R)$	$(1, q_1, R)$
q_1	–	$(1, q_2, R)$
q_2	–	$(1, \text{HALT}, R)$
HALT	–	–

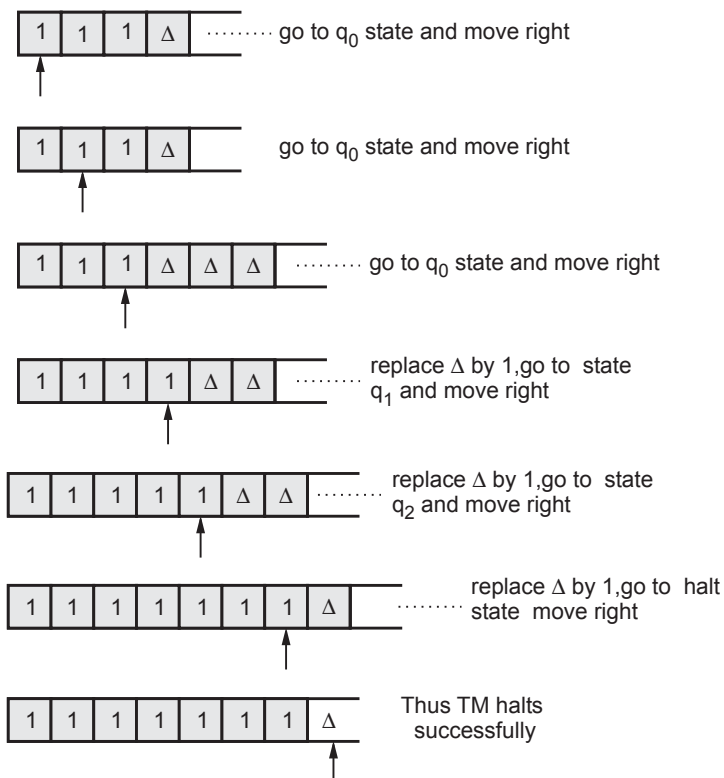


Fig. 5.6.14 (a)

Example 5.6.9 Let T be the turing machine defined by the 5-tuples

$(S_0, 0, 0, S_1, L)$ $(S_0, 1, 0, S_0, L)$
 $(S_0, B, B, \text{Halt}, L)$ $(S_1, 0, 1, S_0, L)$
 $(S_1, 1, 1, S_0, R)$

for each of the following initial tapes, determine the final tape when T halts, assuming that T begins in initial position.

a. 110B b. 0011B c. 0101B

SPPU : May-06, 07, Dec.-07, Marks 12

Solution : The turing machine can be drawn from given δ function, which is as follows -

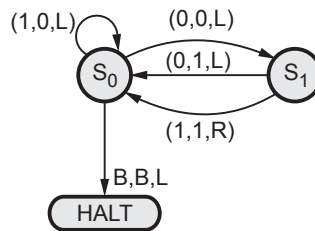


Fig. 5.6.15

- a) 1 1 0 B In S_0 , Convert 1 to 0 and move left.
 ↑
 ↑ 0 1 0 B If we assume B (i.e. blank symbol on the left side i.e. if there is a two way tape) then HALT. Leaving "010" on tape.
- b) 0 0 1 1 B Convert 0 to 0 and move left
 ↑
 ↑ 0 0 1 1 B In state S_1 there is no transition for B. Hence just leave "0011" on the tape without entering the HALT state.
- c) 0 1 0 1 B Convert 0 to 0, move left, go to state S_1 .
 ↑

No transition is possible. Hence leave "0101" on the tape.

Example 5.6.10 Design a turing machine M to implement the function multiplication using the subroutine 'copy'.

Solution : Initially the input is placed on the tape as follows.

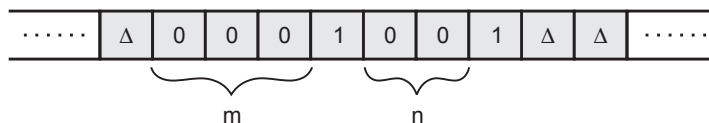


Fig. 5.6.16

The TM can be

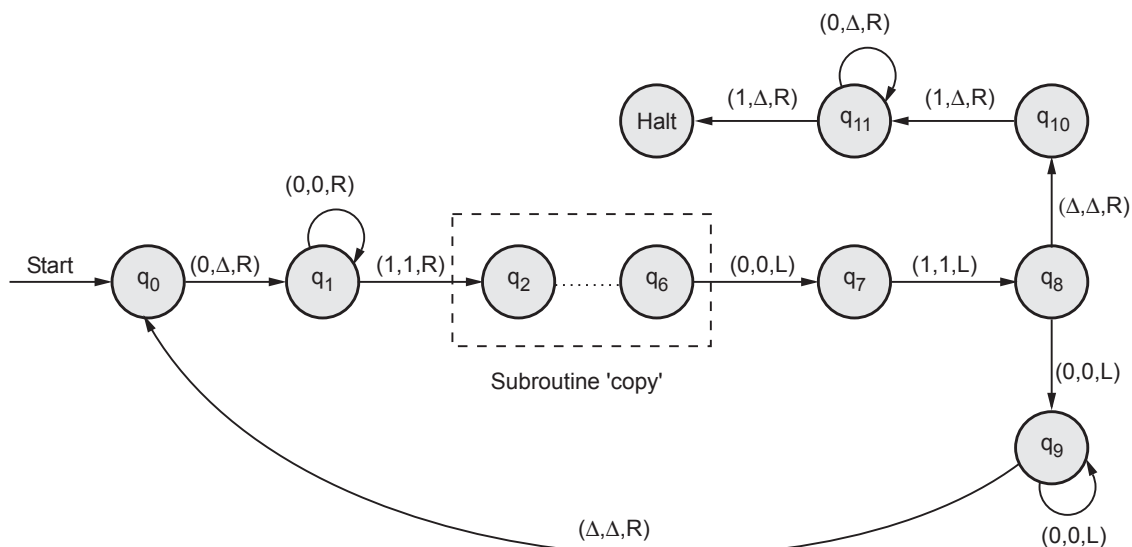


Fig. 5.6.17

The subroutine copy can be as shown below.

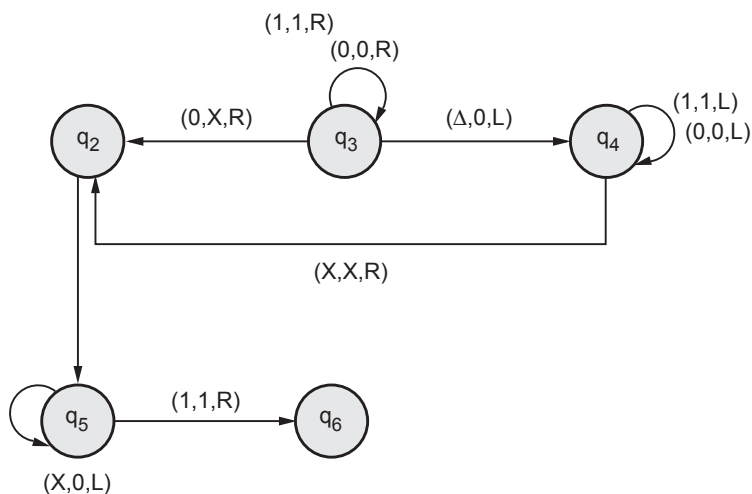


Fig. 5.6.18

To simulate above TM, let us take an example for multiplication of 3 and 2. The inputs are represented by unary 0 and separated by 1.

$\Delta 0001001\Delta \dots$

Convert to Δ and move right.

↑

$\Delta \Delta 0001001\Delta \Delta \Delta \Delta \Delta \dots$

Move right.

↑

Δ Δ 0 0 1 0 0 1 Δ Δ Δ...

↑

Δ Δ 0 0 1 0 0 1 Δ Δ Δ ...

↑

Δ Δ 0 0 1 0 0 1 Δ Δ Δ ...

↑

Δ Δ 0 0 1 X 0 1 Δ Δ Δ ...

↑

Δ Δ 0 0 1 X 0 1 Δ Δ Δ ...

↑

Δ Δ 0 0 1 X 0 1 Δ ...

↑

Δ Δ 0 0 1 X 0 1 0 Δ Δ ...

↑

Δ Δ 0 0 1 X 0 1 0 Δ Δ

↑

Δ Δ 0 0 1 X 0 1 0 Δ Δ

↑

Δ Δ 0 0 1 X X 1 0 Δ Δ

↑

Δ Δ 0 0 1 X X 1 0 Δ Δ

↑

Δ Δ 0 0 1 X X 1 0 Δ

↑

Δ Δ 0 0 1 X X 1 0 Δ

↑

Δ Δ 0 0 1 X X 1 0 0 Δ

↑

Δ Δ 0 0 1 X X 1 0 0 Δ

↑

Move right.

Move right and call 'copy'.

Convert 0 to X and move right.

Move right.

Move right.

Convert to 0 and move left.

Move left.

Move left.

Move right.

Convert 0 to X, move right.

Move right, skip all 0's and 1's.

Convert Δ to 0 and move left.

Skip all 1's and 0's and reach at X by moving left.

Move right.

Move left and convert X to 0's.

$\Delta \Delta 0 0 1 0 0 1 0 0 \Delta$

Move right.

↑

 $\Delta \Delta 0 0 1 0 0 1 0 0 \Delta$

Move left, skip all 0's and 1's

↑

and reach at Δ by moving left.
 $\Delta \Delta 0 0 1 0 0 1 0 0 \Delta$
Move right convert 0 to Δ .

↑

 $\Delta \Delta \Delta 0 1 0 0 1 0 0 \Delta$

↑

Now again perform copy operation as illustrated above. The tape will then consist of

 $\Delta \Delta \Delta 0 1 0 0 1 0 0 0 0 \Delta$
Move right convert 0 to Δ and

↑

repeat the above illustrated steps.

 $\Delta \Delta \Delta \Delta 1 0 0 1 0 0 0 0 0 0$

↑

As we have read out the value of m from $m \times n$ we will convert remaining symbols to Δ leaving only multiplication result on the tape.

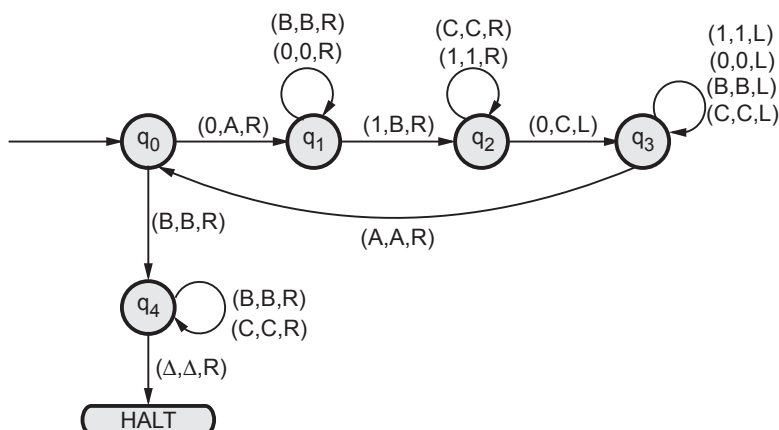
Thus finally $\Delta \Delta \Delta \Delta \Delta \Delta 0 0 0 0 0 \Delta$ HALT

Example 5.6.11 Design turing machine that accepts a language

$$L = \{0^n 1^n 0^n \mid n \geq 1\}.$$

SPPU : May-15, End Sem, Dec.-18, End Sem, Marks 8

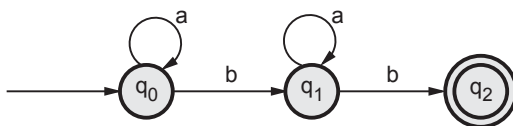
Solution :



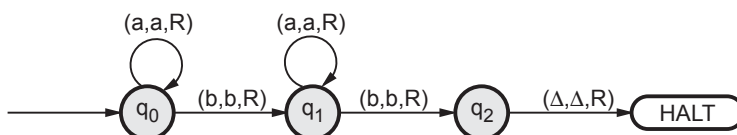
Example 5.6.12 Construct a TM that accepts a language L , a^*ba^*b .

SPPU : May-15, End Sem, Marks 6

Solution : We will construct FA for given L i.e. a^*ba^*b



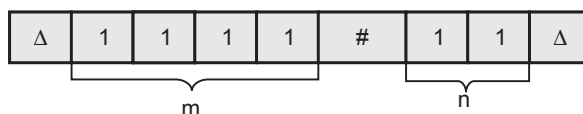
The TM will be,



Example 5.6.13 Design turing machines for each of the following problems : i) Given two unary numbers, m and n , display, 'G', if $m > n$, 'E', if $m = n$, 'L', if $m < n$

SPPU : Dec.-15, End Sem, Marks 10

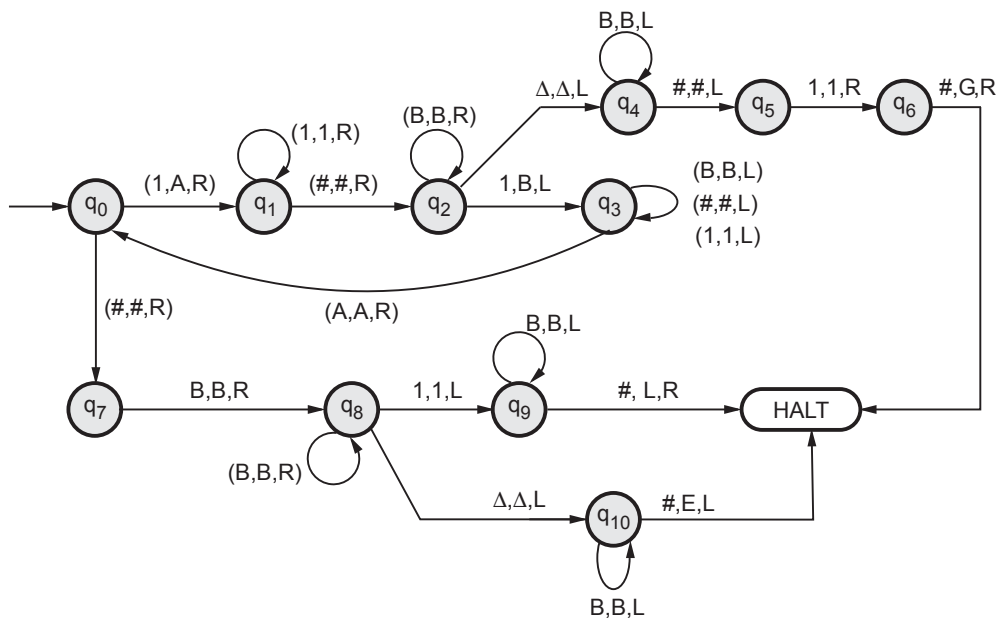
Solution : i) The two unary numbers are placed on the tape and are separated by #. For example



The logic can be as follows :

$m > n$ case	$m < n$ case	$m = n$ case
Δ 1111#11 Δ	Δ 11 # 1111 Δ	Δ 11 # 11 Δ
Δ A111#B1 Δ	Δ A1# B111 Δ	Δ A1 # B1 Δ
Δ AA11#BB Δ	Δ AA # BB11 Δ	Δ AA # BB Δ
Δ AA11GBB Δ	Δ AA L BB11 Δ	Δ AA E BB Δ

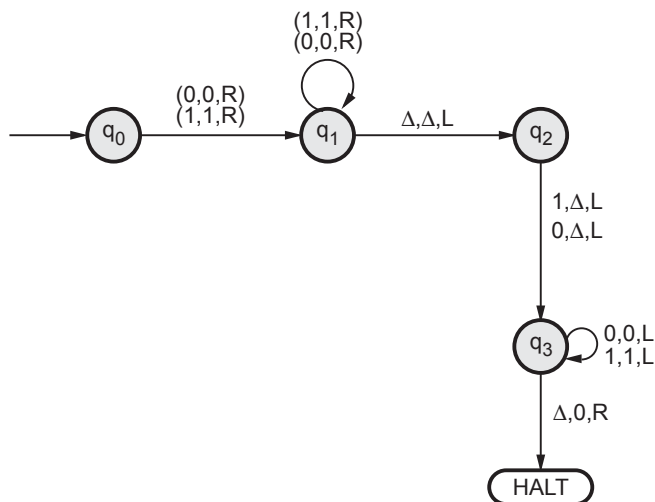
The TM will be -



Example 5.6.14 Design a turing machine to perform right shift operation on a binary number.

SPPU : Dec.-15, End Sem, Marks 6

Solution : In right shift operation the LSB bit is discarded and at MSB the padding 0 is attached. The right shift operation also means division by 2. The TM will be,



Example 5.6.15 Construct a two tape TM to convert an input W into WW^R

SPPU : May-16, End Sem, Marks 6

Solution :

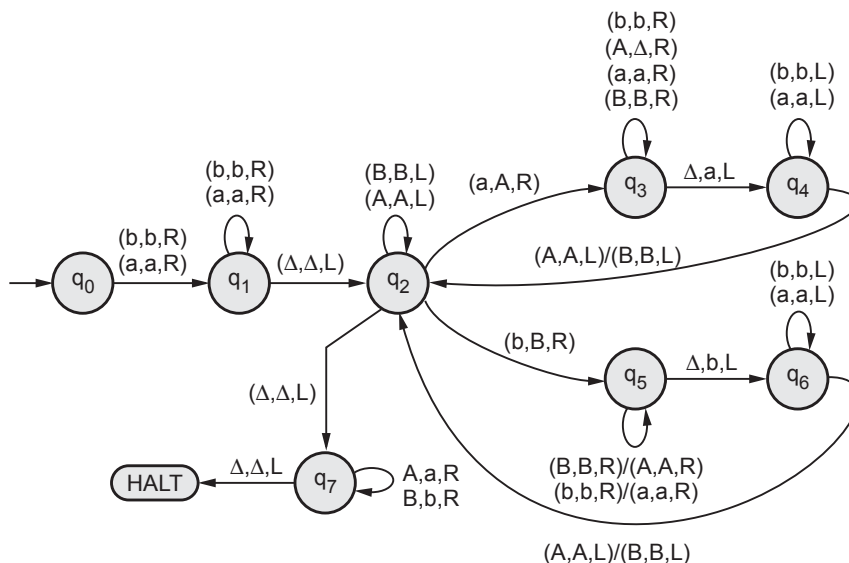


Fig. 5.6.19

Simulation of string $\Delta a b a b \Delta$

$\Delta a b a b \Delta \dots$	Read a, move right towards Δ .
↑	
$\Delta a b a b \Delta$	Read Δ , keep Δ as it is and move left.
↑	
$\Delta a b a b \Delta$	Read b, convert it to B, move right.
↑	
$\Delta a b a B \Delta$	Convert Δ to be and move left.
↑	
$\Delta a b a B b$	Read B as it is and move left.
↑	
$\Delta a b a B b \Delta$	Read a, convert it to A move right.
↑	
$\Delta a b a B b \Delta$	Move towards Δ .
↑	
$\Delta a b A B b \Delta$	Convert Δ to a and move left.
↑	

$\Delta ab AB b a \Delta$ ↑	Keep moving to left by skipping a's or b's.
$\Delta ab AB b a \Delta$ ↑	Read B, move left. Skip all capital letters and move towards left.
$\Delta ab AB b a \Delta$ ↑	Read b, convert it to B and move right.
$\Delta aB AB b a \Delta$ ↑	Go on moving right towards Δ .
$\Delta aB AB b a \Delta$ ↑	Convert Δ to b and move left.
$\Delta aB AB b ab \Delta$ ↑	Keep moving left towards a.
$\Delta aB AB bab \Delta$ ↑	Convert a to A. Keep moving right towards Δ .
$\Delta AB AB b ab \Delta$ ↑	Convert Δ to a
$\Delta AB AB b ab a \Delta$ ↑	Keep moving left towards A.
$\Delta AB AB b ab a \Delta$ ↑	Move left.
$\Delta AB AB b ab a \Delta$ ↑	Move right convert all capitals to small letters.
$\Delta ab ab b ab a \Delta$ ↑	HALT.

5.7 Variants of Turing Machines

SPPU : Dec.-17, Marks 4

There are two types of Turing machines - Deterministic turing machine and non deterministic turing machine.

5.7.1 Deterministic Turing Machine (DTM)

The deterministic Turing machine is a kind of turing machine in which the set of rules denote the single specific action on reading particular input in current specific state. It is just similar to deterministic finite automata.

For example - Refer example 5.5.12.

5.7.2 Non Deterministic Turing Machine (NTM)

The non deterministic turing machine is a kind of turing machine in which the set of rules denote more than one specific action on reading particular input in current specific state. This kind of TM just similar to NFA.

For example -

Note that any language accepted by non deterministic turing machine can also be accepted by deterministic turing machine.

The power of non deterministic turing machine is just similar to the power of deterministic turing machine.

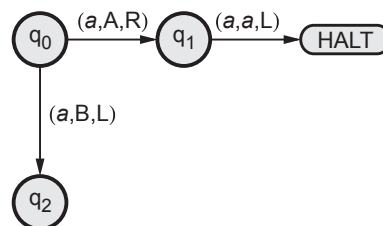


Fig. 5.7.1 NTM

5.7.3 Universal Turing Machine

- The universal language L_u is a set of binary strings which can be modeled by a turing machine.
- The universal language can be represented by a **pair (M, w)** where M is a TM that accepts this languages and w is a binary string in $(0 + 1)^*$ such that w belongs to $L(M)$. Thus we can say that any binary string belongs to a universal language.
- The **universal language** can be represented by $L_u = L(U)$ where U is a universal Turing Machine.
- In fact U is a binary string. This binary string represents various codes of many Turing machines.
- Thus the universal turing machine is a Turing machine which accepts many Turing machines.

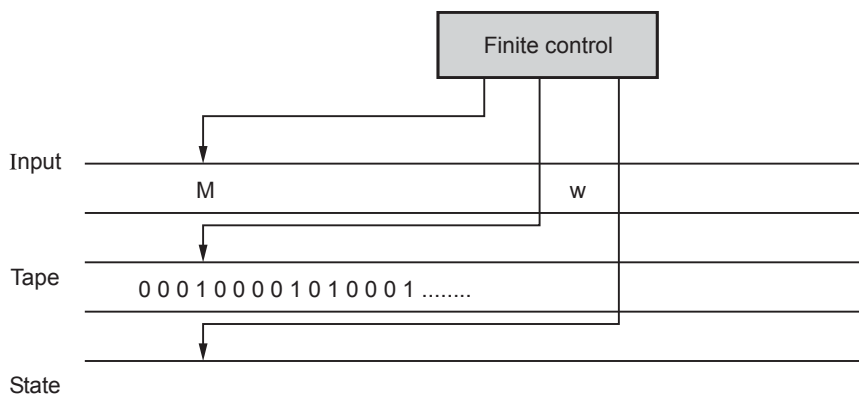


Fig. 5.7.2

5.7.4 Multi-tape Turing Machine

The multitape turing machine is a type of turing machine in which there are more than one input tapes. Each tape is divided into cells and each cell can hold any symbol of finite tape alphabet. The multitape turing machine is as shown in the Fig. 5.7.3.

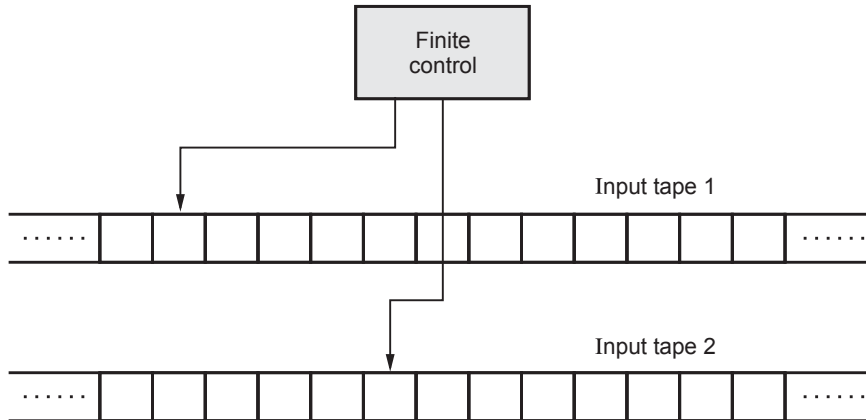


Fig. 5.7.3 A Multitape turing machine

Review Question

1. Write short note on - i) Universal turing machine ii) Multitape turning machine.

SPPU : Dec.-17, End Sem, Marks 4

5.8 Halting Problem of TM

SPPU : May-16,19, Dec.-17,19, Marks 4

Halting Problem can be stated as follows - Given any functional matrix, input data tape and initial configuration, then it is decide whether the process will run forever or will eventually stop."

The halting problem is unsolvable.

Proof : Let, there exists a TM M_1 which decides whether or not any computation by a TM T will ever halt when a description d_T of T and tape t of T is given [That means the input to machine M_1 will be (machine, tape) pair]. Then for every input (t, d_T) to M_1 if T halt for input t , M_1 also halts which is called accept halt. Similarly if T does not halt for input t then the M_1 will halt which is called reject halt. This is shown in Fig. 5.8.1.

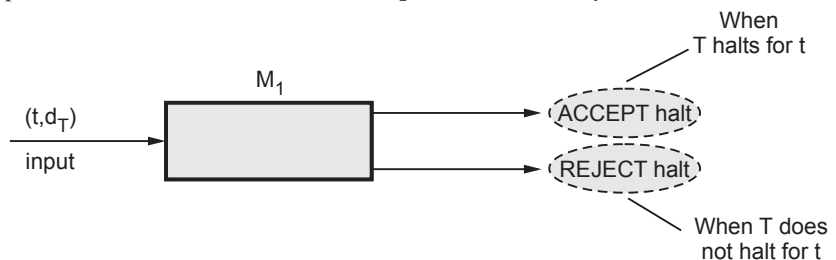


Fig. 5.8.1

Now we will consider another Turing Machine M_2 which takes an input d_T . It first copies d_T and duplicates d_T on its tape and then this duplicated tape information is given as input to machine M_1 . But machine M_1 is a modified machine with the modification that whenever M_1 is supposed to reach an accept halt, M_2 loops forever. Hence behavior of M_2 is as given. It loops if T halts for input $t = d_T$ and halts if T does not halt for $T = d_T$. The T is any arbitrary turing machine.

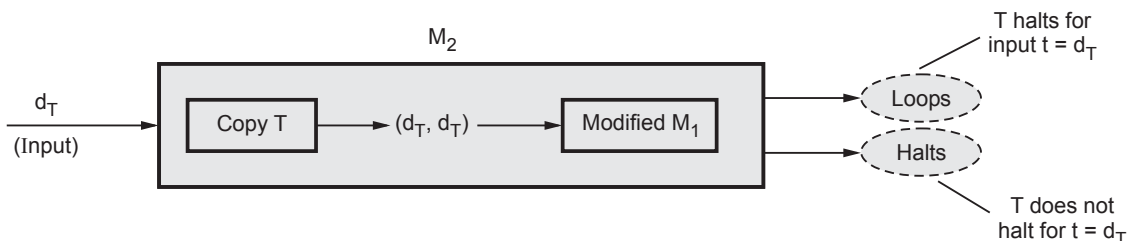


Fig. 5.8.2

As M_2 itself is one turing machine we will take $M_2 = T$. That means we will replace T by M_2 from above given machine.

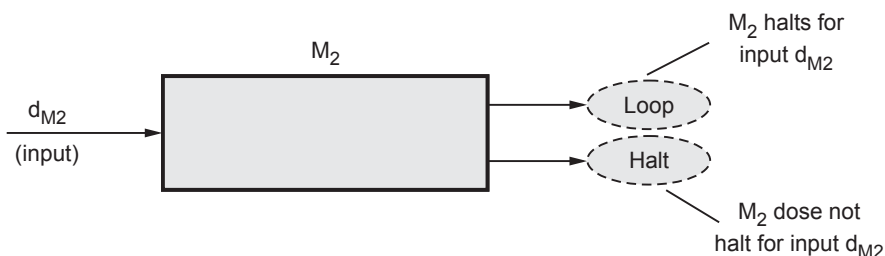


Fig. 5.8.3

Thus machine M_2 halts for input d_{M_2} if M_2 does not halt for input d_{M_2} . This is a contradiction. That means a machine M_1 which can tell whether any other TM will halt on particular input does not exist. Hence halting problem is unsolvable.

Limitations of Turing Machine

Turing machine is a model which can solve any problem which has some algorithm. But not all the problems be solved by TM. There are some problems which cannot be solved by TM. For example : Halting problem cannot be solved by TM. The Halting problem can be stated as follows - "It is not possible to determine whether a TM will ever halt on particular input". Such a problem is called **unsolvable problem**. There some more problems that are not solvable by TM -

- i) Deciding whether two context free grammars are equivalent.
- ii) Whether a given program will loop forever on particular input.

- iii) If two CFGS are given G_1 and G_2 then $L(G_1) \cap L(G_2) = \phi$ is undecidable.
- iv) Whether the language accepted by TM is empty it is not possible for a TM to answer "yes" or "no" for above problems. Hence these problems are unsolvable by TM.

Example 5.8.1 Differentiate between FA and TM.

SPPU : May-19, End Sem, Marks 2

Solution :

Sr. No.	FA	TM
1.	It accepts regular languages.	It accepts recursive and recursively enumerable languages.
2.	It can not be programmed.	It can be programmed.
3.	It accepts small class of language.	It accepts large class of languages.

Review Questions

- Write a short note on : Halting problem.

SPPU : May-16, End Sem, Marks 3, Dec.-19, End Sem, Marks 4

- Write short note on - Limitations of turing machine.

SPPU : Dec.-17, End Sem, Marks 2

- Write short notes on : i) Universal turing machine ii) Multi-tape turing machine

iii) Limitation of turing machine.

SPPU : Dec.-17, End Sem, Marks

5.9 Halting Vs Looping

Halting problem is denoted by following question - "Given a program or algorithm will ever halt or not.

Halting means that the program on certain input will either **accept it and halt** or **reject it and halt**. It would never go into an infinite loop. Basically halting means terminating. So can we have an algorithm that will tell that the given program will halt or not.

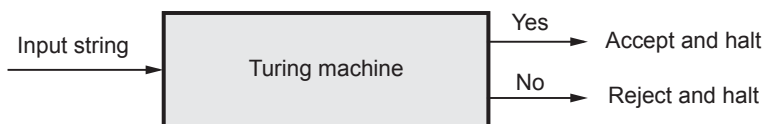


Fig. 5.9.11 Halting

Looping means the program on certain input will either accept it and **enter it in infinite loop** or **reject it and halt**.

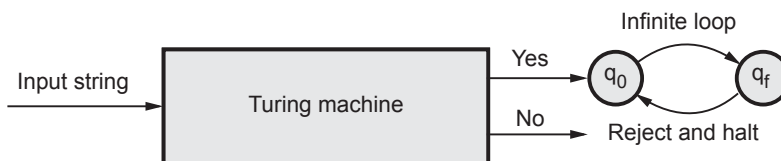


Fig. 5.9.2 Looping

The halting and looping both are undecidable problems.

5.10 A Turing - unrecognizable Language

- Turing machine can accept Type 0 Grammar.
- The type 0 grammar is said to be unrestricted grammar. Examples of unrestricted languages are almost all natural languages.
- The class of languages accepted by Type 0 grammar are called **recursively enumerable languages**.
- The productions can be in the form of $\alpha \rightarrow \beta$ where α is a string of terminals and non-terminals with at least one non-terminal and α cannot be null. β is a string of terminals and non-terminals.

There are two type of languages - 1) Recursive language and 2) Recursively Enumerable (RE) language.

Recursive Languages :

A language is said to be recursive if there exists a Turing machine that **accepts** every string of the language and every string is **rejected** if it is not belonging to that language.

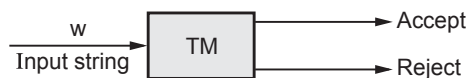


Fig. 5.10.1 TM Accepting Recursive Language

Recursively Enumerable Languages :

A language is said to be recursive if there exists a Turing machine that **accepts** every string belonging to that language. And if the string does not belong to that language then it can cause a Turing machine to enter in an **infinite loop**.

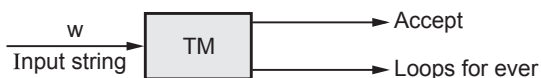


Fig. 5.10.2 TM Accepting RE language

A language is recognizable iff there is a turing machine which will halt and accept only the strings in that language and for strings not in the language, the TM either rejects, or does not halt at all.

Let, the language L is denoted by,

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}$$

is undecidable. That is, there does not exist a turing machine that :

1. Take as input (some encoding of) M and w ,
2. Decide whether M accepts w .

Here L is turing recognizable if there is a turing machine that accepts exactly the words in L , but can either reject or loop indefinitely on an input that is not in L .

To prove that, We will show that $\overline{A_{TM}}$ the complement of A_{TM} , is not turing-recognizable. The proof is by contradiction. Suppose A_{TM} is Turing recognizable, Suppose $\overline{A_{TM}}$ was also Turing recognizable.

Then A_{TM} would be decidable, because we could just make a machine that runs the recognizers for A_{TM} and $\overline{A_{TM}}$ in parallel, and accept if A_{TM} 's recognizer accepts, and rejects if $\overline{A_{TM}}$'s accepts. However, we know that A_{TM} (the famous halting problem) is undecidable, so we have a contradiction. Thus, $\overline{A_{TM}}$ is turing - Unrecognizable.

5.11 Reducibility

- Reducibility is problem solving technique in mathematics.
- Let A and B are two problems and A is reduced to B . If we solve B , we solve A as well. If we can't solve A , we can't solve B .
- For example - If we solve the Eulerian cycle problem, we solve the Eulerian path problem.
- To decide if a problem is solvable or not we use mapping reducibility technique.
- If there is a mapping reduction from language A to language B , we say that language A is mapping reducible to language B .
- Notation used for mapping reducibility is $A \leq M^B$ iff language A is mapping reducible to language B .
- If A is reduced to B and B is decidable, then A is decidable.
- If A is undecidable and reducible to B then B is also undecidable.

5.12 Recursion Theorem

Recursive function is class of functions those are turing computable.

Recursive function theory begins with some very elementary functions that are intuitively effective. Then it provides a few methods for building more complicated functions from simpler functions. That means the computability of given function can be proved using the initial function and some building operations which are few in number.

Initial function : The initial functions are the elementary functions whose values are independent of their smaller arguments. The following functions comprise the class of recursive functions.

The zero function : $z(x) = 0$

The successor function : $s(x) = \text{successor of } x \text{ (roughly , "x+1")}$

The identity function : $\text{id}(x) = x$

The zero functions returns zero regardless of its argument.

The successor functions returns the successor of its argument. Since successorship is a more primitive notion.

The zero and successor functions take only one argument each. But the identity functions is designed to take any number of arguments. When it takes one argument (as above) it returns its argument as its value. When it takes more than one argument, it returns one of them. That means,

$$\text{id}(x, y) = x$$

$$\text{id}(x, y) = y$$

Building operations : We will build more complex functions from the initial set by using only three methods that are,

- i) Composition
- ii) Primitive recursion
- iii) Minimization.

i) Composition

We will start with the successor function,

$$s(x) = x + 1$$

Then we may replace its argument, x , with a function. If we replace the argument, x , with the zero function.

$$z(x)$$

then the result is the successor of zero,

$$s(z(x)) = 1$$

$$s(s(z(x))) = 2 \text{ and so on.}$$

In this way, with the help of the initial functions we can describe the natural numbers. This building operations is called "**composition**". It should be clear that when composition is applied to computable functions, only computable functions will result.

ii) Primitive recursion

The second building operation is called **primitive recursion**. Primitive recursion is a method of defining new functions from old function. The function h is defined through functions f and g by primitive recursion when,

$$h(x, 0) = f(x)$$

$$h(x, s(y)) = g(x, h(x, y))$$

where f and g are known **computable functions**. There are two equations. When h 's second argument is zero, the first equation applies; when it is not zero, we use the second. Use of successor function in second equation enforces the condition that the argument be greater than zero. Hence, the first equation applies in the minimal case and the second applied in every other case.

To solve the function by primitive recursion,

- 1) When the second argument of h is zero, the function is equivalent to some known function f and we compute it;
- 2) Otherwise it is equivalent to some known function g and we compute it.

Thus the function obtained will be computable in nature. For example, we can calculate the factorial function using recursion as :

Initially $1! = 1$ and to calculate $n!$, if we multiply n by $(n - 1)$ then it will generate a nonrecursive series. Instead of that we will multiply n by $(n - 1)!$ We can express the calculation of factorial function by following two equations -

$$1! = 1$$

$$n! = n * (n - 1)$$

A strict definition of the factorial function, $f(n)$ then, consists of these two equations :

$$f(n) = 1 \quad \text{when } n = 1 \quad \dots(5.12.1)$$

$$f(n) = n * (f(n-1)) \quad \text{when } n > 1 \quad \dots(5.12.2)$$

Consider $n = 5$ then using equation (5.12.2) we will get,

$$f(5) = 5 * f(4)$$

$$f(4) = 4 * f(3)$$

$$f(3) = 3 * f(2)$$

$$f(2) = 2 * f(1)$$

By putting the value of equation (5.12.1) for calculating $f(5.12.2)$ we will get,

$$f(2) = 2 * 1 = 2$$

Then $f(3) = 3 * f(2)$

$$= 3 * 2$$

$$= 6$$

Then $f(4) = 4 * f(3)$

$$= 4 * 6$$

$$= 24$$

Then $f(5) = 5 * f(4)$

$$= 5 * 24$$

$$= 120$$

Primitive recursion is like mathematical induction. The first equation defines the basis, and the second defines the induction step.

iii) Minimization

If $g(x)$ is a function that computes the least x such that $f(x) = 0$, then we know that g is computable. And then we can say that g is produced from f by minimization. But we can build g by minimization only if $f(x)$ is already known to be computable.

For example : Suppose we want to obtain least x which makes $f(x) = 0$ then we will try the natural numbers 0,1,2,..... until we reach the first value that gives $f(x) = 0$. Now if such search for x never gets terminated then it is called **unbounded minimization**. And if we obtain such x within some tests then it is called **bounded minimization**. While unbounded minimization has the disadvantages of a partial function which may never terminate, bounded minimization has the disadvantage of sometimes failing to minimize.

5.12.1 Class of Recursive Functions

The classification of recursive functions is as shown in Fig. 5.12.1.

(See Fig. 5.12.1 on next page).

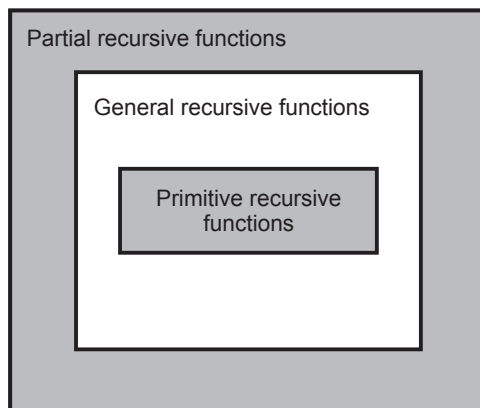


Fig. 5.12.1 Classification of recursive functions

1. Partial Recursive Function

The function is called **partial recursive function** if it can be obtained by applying composition, primitive recursion and minimization as building operations.

2. General Recursive Function

The function is called **general recursive** if it is obtained by applying composition, primitive recursive and an unbounded minimization that happen to terminate. The general recursive function is a larger class than partial recursive functions.

3. Primitive Recursive Function

The function is called **primitive recursive** if it is obtained by applying composition, primitive recursion and unbounded minimization that does not terminate. The set of general recursive function is the same as the set of turing computable functions. The example of general recursive function is an Ackermann's function. The Ackermann's function can be defined as follows -

$$A(0, y) = y + 1$$

$$A(x+1, 0) = A(x, 1)$$

$$A(x+1, y+1) = A(x, A(x+1, y))$$

Then we can calculate $A(x, y)$ for every pair of (x, y) .

Example 5.12.1 Compute $A(1, 1)$, $A(1, 2)$, $A(2, 1)$, $A(2, 2)$ using Ackermann's function.

Solution : Let,

$$A(0, y) = y + 1 \quad \dots (1)$$

$$A(x+1, 0) = A(x, 1) \quad \dots (2)$$

$$A(x+1, y+1) = A(x, A(x+1, y)) \quad \dots (3)$$

By the Ackermann's function,

To compute $A(1, 1)$ put $x = 0$, $y = 0$, then

$$A(1, 1) = A(0 + 1, 0 + 1) \quad \dots \text{Using equation (3)}$$

$$= A(0, A(0 + 1, 0))$$

$$= A(0, A(0, 1)) \quad \dots \text{Using equation (2)}$$

$$= A(A(0, 1)) \quad \dots \text{Using equation (1)}$$

$$= A(0, 2) \quad \dots \text{Using equation (1)}$$

Hence $A(1, 1) = 3$

To compute $A(1, 2)$ put $x = 0$ and $y = 1$

$$A(1, 2) = A(0 + 1, 1 + 1) \quad \dots \text{Using equation (3)}$$

$$= A(0, A(1, 1))$$

$$= A(0, 3) \quad \dots \text{Using equation (1)}$$

$$= 4$$

To compute $A(2, 1)$ put $x = 1$ and $y = 0$

$$A(2, 1) = A(1 + 1, 0 + 1)$$

$$= A(1, A(2, 0))$$

$$= A(1, A(1, 1))$$

$$= A(1, 3)$$

$$= A(0 + 1, 2 + 1)$$

$$= A(0, A(1, 2))$$

$$= A(0, 4)$$

$$= 5$$

To compute $A(2, 2)$ put $x = 1$ and $y = 1$

$$A(2, 2) = A(1 + 1, 1 + 1)$$

$$= A(1, A(2, 1))$$

$$= A(1, 5)$$

Now we will compute $A(1, 5)$ where in $x = 0$ and $y = 4$.

$$A(1, 5) = A(0 + 1, 4 + 1)$$

$$\begin{aligned}
&= A(0, A(1, 4)) \\
&= A(0, (A(0 + 1, 3 + 1))) \\
&= A(0, (A(0, A(1, 3)))) \\
&= A(0, (A(0, A(0 + 1, (2 + 1)))) \\
&= A(0, (A(0, A(0, A(1, 2)))) \\
&= A(0, (A(0, A(0, (4)))) \\
&= A(0, (A(0, 5))) \\
&= A(0, 6)
\end{aligned}$$

$$A(1, 5) = 7$$

Hence $A(2, 2) = A(1, 5)$

$$A(2, 2) = 7$$

5.13 The Model of Linear Bounded Automata

- The Linear Bounded Automata (LBA) is a model which was originally developed as a model for actual computers rather than model for computational process.
- A linear bounded automaton is a restricted form of a non deterministic Turing machine.

Concept : A linear bounded automaton is a multitape Turing machine which has only one tape and this tape is exactly of same length as that of input.

- The Linear Bounded Automaton (LBA) accepts the string in the similar manner as that of Turing machine does.
- For LBA halting means accepting. In LBA computation is restricted to an area bounded by length of the input. This is very much similar to programming environment where size of variable is bounded by its data type.

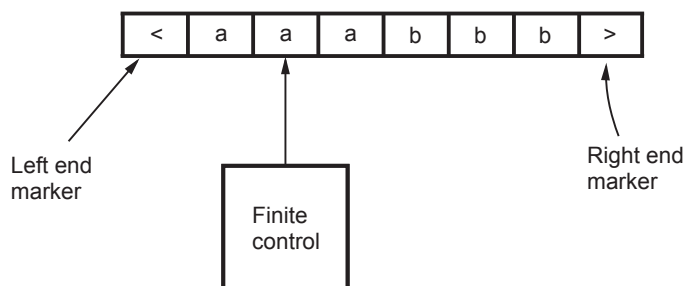


Fig. 5.13.1 Linear bound automaton

- The LBA is powerful than Non deterministic Push Down Automata(NPDA) but less powerful than Turing machine.
- The input is placed on the input tape with beginning and end markers.

In the given Fig. 5.13.1 the input is bounded by < and >.

A linear bounded automata can be **formally defined as** :

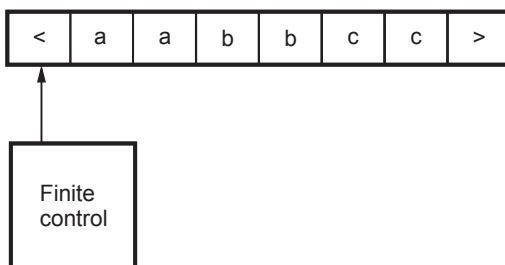
It is 7 - tuple non deterministic Turing machine with

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}) \text{ having}$$

1. Two extra symbols of left end marker and right end marker which are not elements of Γ .
2. The input lies between these end markers.
3. The TM cannot replace < or > with anything else nor move the tape head left of < or right of >.

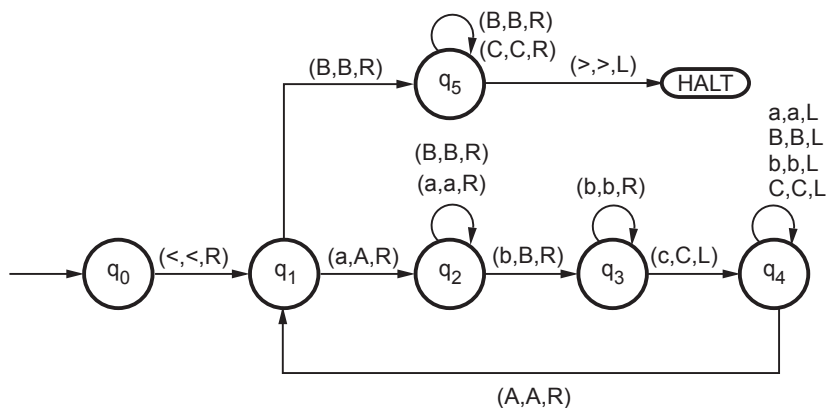
Example 5.13.1 Construct a language $L = \{a^n b^n c^n \mid n \geq 1\}$ using LBA.

Solution :



Step 1 : The input is placed on the input tape which is enclosed within left end marker and right end marker.

Step 2 : We will apply the simple logic as : when we read 'a' convert it to A then move right by skipping all a's. On encountering first 'b' we will convert it to B. Then move right by skipping all b's. On receiving first c convert it to C. Move in left direction unless you get A. Repeat the above procedure and convert equal number of a's, b's, and c's to corresponding A's, B's and C's.- Finally move completely to the rightmost symbol if it is '>' a right end marker, then HALT. The machine will be -

Step 3 :

Simulation : Consider input **aabbcc**

< aabbcc >	Move right.
↑	
< aabbcc >	Convert to A, move right.
↑	
< Aabbcc >	Move right.
↑	
< Aabbcc >	Convert to B, move right.
↑	
< AaBbcc >	Move right.
↑	
< AaBbcc >	Convert to C, move left.
↑	
< AaBbCc >	Move left.
↑	
< AaBbCc >	Move left.
↑	
< AaBbCc >	Move left.
↑	

< AaBbCc >	Move right.
↑	
< AaBbCc >	Convert to A, Move right.
↑	
< AABbCc >	Move right.
↑	
< AABbCc >	Convert to B, move right.
↑	
< AABBCc >	Move right.
↑	
< AABBCc >	Convert to C, move left.
↑	
< AABBCc >	Move left continuously by skipping B's.
↑	
< AABBCc >	Move right
↑	
< AABBCc >	If we get B, we will move right to check whether all b's and c's are converted to B and C.
↑	
< AABBCc >	If we get right end marker '>' then we HALT by accepting the input aabbcc.
↑	

Thus in LBA the length of tape exactly equal to the input string and tape head can not move left of '<' and right of '>'.

Context Sensitive Languages (C.S. Languages)

The context sensitive languages are the languages which are accepted by linear bounded automata. These type of language are defined by context sensitive grammar. In this grammar more than one terminal or non terminal symbol may appear on the left hand side of the production rule. Along with it, the context sensitive grammar follows following rules -

- i) The number of symbols on the left hand side must not exceed number of symbols on the right hand side.
- ii) The rule of the form $A \rightarrow \epsilon$ is not allowed unless A is a start symbol. It does not occur on the right hand side of any rule.

The classic example of context sensitive language is,
 $L = \{a^n b^n c^n | n \geq 1\}$. The context sensitive grammar can be written as -

$S \rightarrow aBC$
 $S \rightarrow SABC$
 $CA \rightarrow AC$
 $BA \rightarrow AB$
 $CB \rightarrow BC$
 $aA \rightarrow aa$
 $aB \rightarrow ab$
 $bB \rightarrow bb$
 $bC \rightarrow bc$
 $cC \rightarrow cc$

Now to derive the string aabbcc we will start from start symbol -

S	rule $S \rightarrow SABC$
<u>S</u> ABC	rule $S \rightarrow aBC$
a <u>BC</u> ABC	rule $CA \rightarrow AC$
aB <u>AC</u> BC	rule $CB \rightarrow BC$
a <u>B</u> ABCC	rule $BA \rightarrow AB$
a <u>A</u> BBCC	rule $aA \rightarrow aa$
aa <u>B</u> BCC	rule $aB \rightarrow ab$
aab <u>B</u> CC	rule $bB \rightarrow bb$
aabb <u>C</u> C	rule $bC \rightarrow bc$
aabb <u>c</u> C	rule $cC \rightarrow cc$
aabbcc	

Note that the language $a^n b^n c^n$ where $n \geq 1$ is represented by context sensitive grammar but it can not be represented by context free grammar.

Every contentext sensitive language can be represented by LBA.

5.14 Fill in the Blanks

- Q.1** The language accepted by Turing machine is called ____ .
- Q.2** Recursive is synonym to ____ .
- Q.3** The complement of recursive language is ____ .
- Q.4** NP stands for ____ .
- Q.5** P stands for ____ .
- Q.6** Searching the element is done in ____ time.
- Q.7** Kruskal's algorithm is done in ____ time.
- Q.8** ____ and ____ is said to be NP complete.
- Q.9** PCP stands for ____ .

5.15 Multiple Choice Questions

- Q.1 Which of the following problems is undecidable ?**
- | | |
|--|---|
| ☐ a Membership problem for CFGs | ☐ b Ambiguity problem for CFGs |
| ☐ c Finiteness problem for FSAs | ☐ d Equivalence problem for FSAs |
- Q.2 A language L is said to be _____ if there is a turing machine M such that $L(M) = L$ and M halts at every point.**
- | | |
|--|--|
| ☐ a turing acceptable | ☐ b decidable |
| ☐ c undecidable | ☐ d none of the mentioned |
- Q.3 $DFA = \{M \text{ is a DFA that recognizes input } w\}$. This language is ____ .**
- | | |
|---------------------------------------|--|
| ☐ a decidable | ☐ b undecidable |
| ☐ c non finite | ☐ d none of these |
- Q.4 A problem is called _____ if its has an efficient algorithm for itself.**
- | | |
|--|--|
| ☐ a tractable | ☐ b intractable |
| ☐ c computational | ☐ d none of the mentioned |
- Q.5 A PCP problem is solvable if ____ .**
- | | |
|------------------------------------|------------------------------------|
| ☐ a $? =1$ | ☐ b $? =2$ |
| ☐ c $? =4$ | ☐ d $? =8$ |

Q.6 Traveling salesman problem belong to _____.

- ☐ a P
 ☐ b NP
☐ c linear
 ☐ d none of these

Q.7 If PCP is decidable then MPCP is _____.

- ☐ a decidable
 ☐ b undecidable
☐ c can't say
 ☐ d none

Answer Keys for Fill in the Blanks :

Q.1	recursive enumerable and recursive	Q.2	decidability
Q.3	recursive	Q.4	Nondeterministic Polynomial time
Q.5	Polynomial time	Q.6	polynomial time
Q.7	nonpolynomial time	Q.8	NP, P
Q.9	Post Correspondence Problem		

Answer Keys for Multiple Choice Questions :

Q.1	b	Q.2	b	Q.3	a	Q.4	a
Q.5	a	Q.6	b	Q.7	a		

